MANCHESTER 1824
The University of Manchester

Energy-Efficient UAV Path Planning in IOT Networks: A Deep Reinforcement Learning Aided Approach

A dissertation submitted to The University of Manchester for the degree of

Bachelor of Engineering in Electrical and Electronics Engineering

in the Faculty of Science and Engineering

Year of submission

2026

Student ID

11085980

School of Engineering

Contents

Contents **2**

Abstract **7**

Declaration of originality **8**

Copyright statement **9**

Acknowledgments **10**

Terms and Abbreviations **11**

1 Introduction **12**

1.1 Background and motivation 12

1.2 Aims and objectives 13

1.3 Report structure 14

2 Literature Review **14**

2.1 LoRaWAN Scalability and the “SF Monoculture” 14

2.1.1 The SF Monoculture and ADR Failure 15

2.1.2 The Capture Effect as a Partial Remedy 15

2.2 Existing Optimisation Approaches 16

2.2.1 Algorithm Selection: DQN versus Policy-Gradient Methods 17

2.3 The Shift to AI-Native Networking 18

2.3.1 Deep Q-Networks in Communications 18

2.3.2 DRL for UAV Path Planning 18

2.3.3 Architecture Selection: MLP versus Attention and Graph Neural Networks . . . 19

2.3.4 Information Freshness and Domain Generalisation 20

2.4 Summary and Project Positioning 20

2.4.1 Simulation Tool Gap 20

3 Preliminaries **22**

3.1 LoRaWAN Physical Layer 22

3.1.1 Spreading Factors and Orthogonality 22

3.1.2 The Capture Effect 22

3.2 Channel Propagation Model 22

3.3 Deep Reinforcement Learning . . . 23
3.3.1 Markov Decision Process . . . 23
3.3.2 Partially Observable Markov Decision Process . . . 24
3.3.3 Deep Q-Networks (DQN) . . . 25

4 Methodology . . . 25
4.1 System Model . . . 25
4.1.1 UAV Kinematics and Energy Consumption . . . 27
4.1.2 Channel Model and Protocol Modelling . . . 28
4.1.3 Device Class and Traffic Model . . . 28
4.1.4 Collision Resolution via Capture Effect . . . 28
4.1.5 Gateway Architecture . . . 30
4.1.6 Link Budget, Collection Radius, and Operational Envelope . . . 31
4.2 Problem Formulation . . . 32
4.2.1 State Space Definition . . . 33
4.2.2 Action Space Definition . . . 34
4.3 Sensor Placement in Simulation . . . 34
4.3.1 Stochastic Sensor Deployment . . . 34
4.3.2 Multi-Objective Reward Function . . . 34
4.4 Deep Reinforcement Learning Implementation . . . 36
4.4.1 Deep Q-Network (DQN) Architecture . . . 36
4.4.2 Probabilistic Link-Layer Model . . . 36
4.4.3 Training Convergence . . . 37
4.5 Architecture and Interface . . . 37
4.5.1 Training Algorithm . . . 37
4.5.2 Hyperparameters . . . 38
4.6 Summary . . . 38

5 Results and Discussion . . . 38
5.1 Experimental Setup . . . 38
5.2 Single-Episode Trajectory Analysis . . . 39
5.2.1 Spatial Trajectory Comparison . . . 39
5.2.2 Spreading Factor Distribution Over Time . . . 40
5.3 Multi-Condition Comparative Analysis . . . 40
5.3.1 Sensor-Count Scaling at 500×500 . . . 40

5.3.2 Statistical Summary . . . 42
5.4 Operational Envelope: Grid-Scale Analysis . . . 42
5.5 Ablation Study . . . 44
5.6 Robustness and Generalisation Analysis . . . 46
5.6.1 Sim-to-Real Perturbation Tests . . . 46
5.6.2 Cross-Layout Generalisation . . . 47
5.6.3 Clustered Layout Curriculum: A Negative Result . . . 47
5.6.4 Hardware-in-the-Loop Preparation Analysis . . . 48
5.7 Discussion . . . 49
5.7.1 Statistical Parity with Greedy Baselines . . . 49
5.7.2 The Operational Envelope . . . 49
5.7.3 Generalisation and the Bimodal Failure Boundary . . . 50
5.7.4 Limitations . . . 50

6 Conclusions and Future Work . . . 50
6.1 Conclusions . . . 50
6.2 Future Work . . . 52

7 Project Management . . . 53

References . . . 53

Appendices . . . 57

A Project Outline . . . 57

B Project Plan and Gantt Chart . . . 59

C Risk Register . . . 61

D Health & Safety Risk Assessment . . . 63

E Continuing Professional Development (CPD) Log . . . 64

F Code Repository . . . 69

G DQN Hyperparameters . . . 72

H Sim-to-Real Perturbation Results . . . 74

I Additional Results Figures . . . 76

Word count: 12537

Abstract

Mobile UAV gateways in dense LoRaWAN networks face a collision problem: as the UAV moves, rapid link-quality changes cause sensors to converge on identical Spreading Factors, collapsing packet delivery in a phenomenon termed the "SF Monoculture." Existing solutions are siloed, standard path planners ignore this failure mode, and static protocol optimisers cannot adapt to UAV mobility. This dissertation addresses both gaps by treating UAV physical position as a direct control input for the LoRaWAN radio protocol, and by characterising the physical operational envelope within which a single UAV can feasibly collect data.

A protocol-aware Deep Reinforcement Learning (DRL) framework trains a Deep Q-Network (DQN) agent within a custom Gymnasium environment, to the author's knowledge the first open simulation to model the complete causal chain from UAV position through RSSI, EMA-based Adaptive Data Rate (ADR) latency, and the LoRa Capture Effect to packet delivery within a single step. The environment incorporates the Two-Ray Ground Reflection path loss model, log-normal shadowing, and asynchronous duty cycles, producing a realistic link-budget constraint: sensors beyond approximately 365 grid units ($3{,}650$ m) fall outside the collection radius, making UAV positioning genuinely consequential. A Domain Randomisation and Curriculum Learning regime produces a single generalised policy deployable without retraining across grids from 100×100 to $1{,}000\times1{,}000$ cells and network densities of 10 to 40 sensors.

Multi-seed evaluation (20 seeds per condition) against a protocol-blind Nearest-Sensor Greedy and a novel SF-Aware Max-Throughput Greedy baseline reveals a clear operational envelope. At compact scales (100×100, 300×300), all sensors remain within link-budget range from any UAV position and all agents achieve near-identical coverage. At intermediate scales (500×500), link-budget constraints become active: the DQN's protocol-aware repositioning strategy yields statistically significant throughput advantages over both greedy baselines ($p < 0.05$, Welch's t-test). At $1{,}000\times1{,}000$, the problem becomes physically infeasible for a single UAV: the energy budget supports at most $2{,}100$ steps of flight, insufficient to traverse the $14{,}142$ m grid diagonal, and no agent achieves full sensor coverage. The key emergent behaviour is strategic dwell-then-reposition: the DQN hovers near sensor clusters to drive EMA-ADR convergence to lower Spreading Factors before repositioning to exploit orthogonal-SF concurrent reception, a temporal planning strategy that one-step greedy heuristics cannot replicate. A component ablation confirms that the Capture Effect and Domain Randomisation are individually necessary, while the physical infeasibility at

$1{,}000 \times 1{,}000$ scale identifies multi-UAV coordination as the required architectural extension.

Declaration of originality

I hereby confirm that no portion of the work referred to in the thesis has been submitted in support of an application for another degree or qualification of this or any other university or other institute of learning.

Copyright statement

i The author of this thesis (including any appendices and/or schedules to this thesis) owns certain copyright or related rights in it (the "Copyright") and s/he has given The University of Manchester certain rights to use such Copyright, including for administrative purposes.

ii Copies of this thesis, either in full or in extracts and whether in hard or electronic copy, may be made *only* in accordance with the Copyright, Designs and Patents Act 1988 (as amended) and regulations issued under it or, where appropriate, in accordance with licensing agreements which the University has from time to time. This page must form part of any such copies made.

iii The ownership of certain Copyright, patents, designs, trademarks and other intellectual property (the "Intellectual Property") and any reproductions of copyright works in the thesis, for example graphs and tables ("Reproductions"), which may be described in this thesis, may not be owned by the author and may be owned by third parties. Such Intellectual Property and Reproductions cannot and must not be made available for use without the prior written permission of the owner(s) of the relevant Intellectual Property and/or Reproductions.

iv Further information on the conditions under which disclosure, publication and commercialisation of this thesis, the Copyright and any Intellectual Property and/or Reproductions described in it may take place is available in the University IP Policy (see `http://documents.manchester.ac.uk/DocuInfo.aspx?DocID=24420`), in any relevant Thesis restriction declarations deposited in the University Library, The University Library's regulations (see `http://www.library.manchester.ac.uk/about/regulations/`) and in The University's policy on Presentation of Theses.

Acknowledgments

I am grateful to my supervisor Dr. Zahra Mobini for her guidance and support throughout the project.

I would like to thank my family for their support.

Use of AI Tools. AI-assisted writing tools (Claude, Anthropic) were used during the preparation of this dissertation to assist with grammar checking, sentence restructuring, and clarity editing of draft text. All technical content, experimental design, results, analysis, and conclusions are the author's own work. No AI tool was used to generate technical conclusions or to produce any results, figures, or code presented in this report.

Terms and Abbreviations

Acronym	Definition
UAV	Unmanned Aerial Vehicle
IoT	Internet of Things
RL	Reinforcement Learning
DRL	Deep Reinforcement Learning
DQN	Deep Q-Network
PPO	Proximal Policy Optimization
MDP	Markov Decision Process
POMDP	Partially Observable Markov Decision Process
LoRa	Long Range
LoRaWAN	Long Range Wide Area Network
SF	Spreading Factor
ADR	Adaptive Data Rate
RSSI	Received Signal Strength Indicator
SNR	Signal-to-Noise Ratio
PHY	Physical Layer
MAC	Medium Access Control
AoI	Age of Information
DR	Domain Randomization
GPS	Global Positioning System
ToA	Time on Air
GtA	Ground-to-Air
LoS	Line-of-Sight
NLOS	Non-Line-of-Sight
2D	Two-Dimensional
3D	Three-Dimensional

1 Introduction

1.1 Background and motivation

IoT sensor networks support a wide range of applications in smart agriculture, environmental monitoring, and disaster response [1]. In many of these scenarios, fixed infrastructure is either unavailable or prohibitively expensive to deploy. UAVs provide on-demand data collection, serving as mobile gateways deployable to remote or hazardous locations [1], [2], [3]. Combining the long-range, low-power capabilities of the LoRaWAN protocol [4], [5] with UAV mobility allows data collection from vast sensor fields with minimal infrastructure [6].

However, integrating mobile UAVs into dense LoRaWAN networks introduces challenges that standard protocols cannot address. The LoRaWAN architecture, specifically its ADR mechanism, is engineered for static, ground-based networks. As identified by Zorbas et al. (2025), in dense deployments where link qualities change rapidly, such as with a moving UAV, the standard network management fails. Devices often converge to the same transmission settings, leading to a high rate of “intra-SF collisions” and a degradation of network performance [7]. This phenomenon, often referred to as the “SF12 Well” or “SF Monoculture,” causes a congestion collapse where the network’s concurrent channel capacity is wasted, and packet delivery rates plummet.

Existing solutions to this problem remain siloed. Standard path planning heuristics (such as “lawnmower” patterns) optimise solely for flight distance, ignoring the underlying telecommunications physics [8]. These “protocol-blind” trajectories force sensors into the SF Monoculture, causing the UAV to hover while waiting for data that cannot be decoded due to collisions. Conversely, mathematical optimisation approaches (like Mixed Integer Linear Programming) can solve the resource allocation problem for static networks but lack the real-time adaptability required for dynamic, mobile environments [7].

Resolving these failures requires a control framework that treats the UAV’s physical mobility and the LoRaWAN protocol constraints as a joint optimisation problem. Zorbas et al. [7], Nguyen et al. [9], Luong et al. [10], and Challita et al. [11] all converge on reinforcement learning as the right tool for dynamically evolving network conditions: it adapts online without requiring stationary channel assumptions. Using the UAV’s 3D position as a direct control input for the radio protocol lets an agent break the SF Monoculture and recover the channel capacity that greedy trajectories waste.

1.2 Aims and objectives

This project develops a protocol-aware Deep Reinforcement Learning (DRL) framework that optimises UAV flight trajectories for energy-efficient data collection in dense, unsynchronised LoRaWAN networks. The central claim is that using the UAV's physical position as a direct control input for the radio protocol resolves the "Spreading Factor (SF) Monoculture" bottleneck. A Deep Q-Network (DQN) agent is trained within a high-fidelity simulation to learn a network-aware policy that maximises data throughput while minimising energy consumption.

The project objectives are:

- To develop a high-fidelity simulation environment compatible with the Gymnasium interface, incorporating rigorous telecommunications physics including the Two-Ray Ground Reflection path loss model, log-normal shadowing, and asynchronous duty cycles. To the author's knowledge, this is the first open framework to model the complete causal chain from UAV position through RSSI, EMA-ADR, and Capture Effect to packet delivery within a single simulation step.
- To model the specific constraints of the LoRaWAN protocol, particularly the Adaptive Data Rate (ADR) latency, to replicate the causal link between UAV positioning, RSSI, and Spreading Factor allocation.
- To formulate the UAV data collection problem as a Partially Observable Markov Decision Process (POMDP, where sensor locations and out-of-range channel states are hidden from the agent), and to approximate the resulting POMDP as a tractable MDP via frame stacking ($k{=}4$), designing a multi-objective reward function that balances the competing goals of energy conservation, data freshness (AoI), and collection volume.
- To train a Deep Q-Network (DQN) agent to learn an emergent flight policy that autonomously navigates the "Starvation vs. Throughput" trade-off without human intervention.
- To evaluate the protocol-aware DRL agent against two greedy path-planning baselines (Nearest-Sensor Greedy and SF-Aware Max-Throughput Greedy) in terms of energy efficiency, fairness (Jain's Index), and total bytes delivered, using multi-seed statistical testing (Welch's t-test, Cohen's d).

- To characterise the operational envelope of the single-UAV system by evaluating across grid scales from 100×100 to $1{,}000 \times 1{,}000$ cells, identifying the physical feasibility boundary beyond which link-budget and energy constraints prevent full sensor coverage regardless of policy quality.
- To assess the trade-offs between computational complexity, policy robustness, and real-world feasibility, identifying the "Sim-to-Real" gap and the transition from feasible to infeasible regimes as key findings for future research.

A protocol-aware agent generates energy-efficient trajectories satisfying network fairness and throughput requirements without pre-specified routing heuristics.

1.3 Report structure

This report is organised as follows. Section 2 reviews UAV-assisted data collection, LoRaWAN scalability, and DRL in wireless networks. Section 3 covers prerequisite concepts: the LoRaWAN physical layer, Two-Ray channel model, and Q-Learning fundamentals. Section 4 presents the system model, Gymnasium environment, and DRL formulation. Section 5 reports multi-seed comparative performance, ablation studies, robustness analysis, and a discussion of mechanisms. Conclusions and future work are in Section 6.

2 Literature Review

UAVs extend data collection to remote and hazardous environments where static infrastructure is impractical. Wei et al. [1] survey UAV-assisted data collection and identify three structural advantages over fixed gateways: flexible deployment, line-of-sight communication links, and on-demand servicing of sparse sensor networks. The transition from static to mobile gateways, however, introduces complex challenges in network management and protocol stability.

2.1 LoRaWAN Scalability and the "SF Monoculture"

LoRaWAN is the de facto standard for long-range, low-power IoT connectivity [4], [5]. Augustin et al. [4] provide a foundational analysis of the LoRa physical layer, characterising the chirp

spread-spectrum modulation, the six orthogonal spreading factors (SF $\in \{7, \ldots, 12\}$), and the trade-off between data rate and link budget that governs SF selection. Bor et al. [5] demonstrate through large-scale simulation that a single LoRaWAN gateway can theoretically serve thousands of devices, but that this capacity is highly sensitive to the SF distribution: if too many devices converge on the same high-SF setting, per-device throughput collapses due to intra-SF collisions and the resulting long Time-on-Air. These two foundational works establish the protocol constraints that every LoRaWAN system designer, and by extension every UAV mission planner, must respect.

2.1.1 The SF Monoculture and ADR Failure

A critical failure mode in mobile LoRaWAN networks is the congestion collapse known as the "Spreading Factor (SF) Monoculture" or "SF12 Well." Zorbas et al. [7] provide a rigorous mathematical analysis of this phenomenon, demonstrating that the standard LoRaWAN Adaptive Data Rate (ADR) mechanism, which adjusts each device's SF based on a smoothed RSSI history, fails systematically in dense deployments. Because ADR reacts to past link quality rather than current conditions, devices in a rapidly changing environment (such as one serviced by a moving UAV) accumulate outdated RSSI estimates. This latency in the ADR control loop causes devices to converge on high SF settings long after the UAV has moved away, eventually driving the entire network toward SF12, the slowest, longest Time-on-Air setting. The consequence is a self-reinforcing congestion collapse: high SF increases collision probability, reducing successful transmissions, which in turn prevents devices from receiving ADR downgrade commands. Zorbas et al. conclude that "reinforcement learning models could be employed to adapt over time to evolving network conditions," a direct motivation for the DRL approach in this dissertation.

Reynders et al. [12] provide complementary evidence: collision probability grows super-linearly beyond a critical load threshold, and frequency-adaptive rate management is *necessary* to sustain performance, motivating the ADR modelling choices in this work.

2.1.2 The Capture Effect as a Partial Remedy

An important physical mitigation mechanism in congested LoRaWAN networks is the LoRa Capture Effect. Croce et al. [13] are the first to rigorously characterise this phenomenon through controlled software-defined radio (SDR) experiments. They demonstrate experimentally that collisions

between two LoRa signals on the *same* spreading factor can be resolved if the stronger signal exceeds the weaker by a power ratio of approximately 6 dB, the "capture threshold." Below this threshold, both packets are lost; above it, the stronger packet is decoded successfully. This finding overturns a common simplifying assumption in prior LoRaWAN simulators (that any intra-SF collision results in mutual destruction) and has significant implications for capacity analysis.

Croce et al. further show that, contrary to intuition, allocating high SFs to far-away devices (to compensate for path loss) can actually *reduce* cell capacity in congested deployments, because high-SF packets occupy the channel for longer, increasing the probability that a second device transmits during the same Time-on-Air window. This creates a direct tension between coverage (which demands high SFs at range) and capacity (which demands low SFs for throughput). A mobile UAV that can close the physical distance to sensors, reducing path loss and enabling SF downgrade via ADR, can resolve this tension dynamically, something no static gateway can achieve.

This work directly incorporates the 6 dB threshold into the collision resolution logic, rewarding the agent for reaching spatial positions that create the power differential required for capture, a design distinction from all protocol-blind trajectory planners.

2.2 Existing Optimisation Approaches

Zorbas et al. [7] propose a Mixed Integer Linear Programming (MILP) framework to determine the optimal SF distribution that maximises packet reception probability. While this method provides a theoretically optimal allocation for static networks, it has two critical limitations for the mobile-gateway setting: first, MILP scales exponentially with the number of binary SF assignment variables, becoming intractable for $N > 20$ sensors; second, and more critically, it assumes channel conditions are known and stationary at solve time, an assumption that fails continuously as the UAV moves and the EMA-ADR control loop propagates outdated RSSI estimates across the network. The authors themselves acknowledge this limitation, concluding that reinforcement learning is the appropriate tool for the dynamic case [7].

Xiong et al. [8] introduce "FlyingLoRa," which minimises energy consumption by jointly optimising the UAV's 3D trajectory and transmission scheduling. FlyingLoRa optimises offline and requires full knowledge of sensor locations and link budgets before the mission; it omits ADR dynamics (treating each sensor's data rate as fixed) and does not model the Capture Effect, assuming mutual packet destruction on any intra-SF collision. FlyingLoRa therefore cannot exploit the positioning-induced

SF diversity that the protocol-aware DRL agent learns to generate.

Channel and trajectory optimisation surveys (Zeng et al. [3], Mozaffari et al. [2]) establish the channel geometry assumptions underpinning the Two-Ray Ground Reflection model used here, and Zeng and Zhang [14] motivate the energy-consumption term in our reward function. However, none of these works jointly models the physical channel *and* the MAC-layer ADR feedback loop; their positioning conclusions do not account for the SF convergence dynamics that dominate throughput in dense deployments.

At the intersection of deep reinforcement learning and UAV communications, Challita et al. [11] propose a deep RL framework for interference-aware path planning of cellular-connected UAVs, demonstrating that DRL agents can learn collision-avoidance behaviour that static planners cannot replicate, directly analogous to the SF-monoculture avoidance objective here. However, Challita et al. operate in the cellular domain where the interference model is well-characterised and stationary; they do not encounter the non-stationary MAC-layer dynamics (EMA-ADR, duty cycles, Capture Effect) that make the LoRaWAN setting harder. Nemer et al. [15] apply a distributed actor–critic approach to maximise coverage fairness for multi-UAV networks, confirming that Jain's Fairness Index is an appropriate target metric for DRL-based UAV controllers. A limitation of Nemer et al. is that their evaluation uses fixed sensor positions and a simplified channel model, leaving open whether the fairness gains transfer to the stochastic, topology-varying conditions that domain randomisation is designed to address.

2.2.1 Algorithm Selection: DQN versus Policy-Gradient Methods

Three candidate families are considered: DQN [16], PPO [17], and SAC [18]. The UAV's five-action discrete space directly favours DQN: it was designed for discrete actions and achieves sample efficiency via experience replay and a target network. PPO handles discrete actions but is on-policy, discarding all experience after each update, which is costly when episodes span 2100 steps and interactions are computationally expensive. SAC's discrete variant [19] loses the entropy-maximisation guarantee that motivates its exploration, offering no clear advantage here.

Experience replay also stabilises the Q-function as the reward landscape shifts with ADR convergence, a property on-policy PPO cannot replicate. Challita et al. [11] and Luong et al. [10] both support DQN for discrete-waypoint UAV navigation, motivating it as the primary algorithm.

2.3 The Shift to AI-Native Networking

Nguyen et al. [9] outline the 6G vision of AI-native networks where DRL manages massive, heterogeneous IoT connectivity in real time, conditions under which hand-crafted optimisation policies cannot scale.

2.3.1 Deep Q-Networks in Communications

The Deep Q-Network [16] stabilises Q-learning with experience replay and a periodically frozen target network. Experience replay breaks the temporal correlation between consecutive UAV observations (important given the spatial correlation of the flight path), while the target network prevents the Q-value feedback loop that causes divergence. Luong et al. [10] survey DRL in communications and observe that reward shaping and state-space design are critical: poorly chosen representations prevent the agent from learning the temporal patterns needed to anticipate future congestion. This directly motivates the frame-stacking and urgency-proxy design choices in Section 4.

The non-stationary reward landscape introduced by EMA-ADR convergence presents a particular stability challenge that on-policy methods struggle to handle. Because the same state–action pair yields different rewards depending on the accumulated ADR history, on-policy algorithms such as PPO discard relevant transitions before the ADR feedback loop has resolved. DQN's experience replay retains these transitions across ADR timescales, allowing the agent to correlate dwell behaviour with the delayed SF-downgrade reward that follows, a temporal gap of approximately $1/\lambda = 10$ steps that one-step lookahead cannot bridge. This property is central to why off-policy value-based learning is preferred over policy-gradient alternatives in the LoRaWAN setting.

2.3.2 DRL for UAV Path Planning

No prior work applies DRL specifically to UAV path planning in LoRaWAN networks with explicit ADR and Capture Effect modelling. The closest precedents (Challita et al. [11], Nemer et al. [15]) operate in the cellular domain where interference models are stationary and the non-stationary EMA-ADR feedback loop is absent. This gap is the primary motivation for this dissertation.

A further structural difference from prior DRL path-planning work is the coupling between physical positioning and MAC-layer protocol state. In cellular interference avoidance, the interference field

is determined solely by geometry and can be estimated from a single observation. In LoRaWAN, the SF assignment of each sensor depends on its RSSI history filtered through the EMA-ADR loop: the same geometric configuration can yield entirely different SF distributions depending on the UAV's prior trajectory. This history-dependence violates the Markov property and means the agent must infer latent protocol state from temporal sequences rather than instantaneous observations, a qualitatively harder learning problem than the stationary interference settings considered by Challita et al. and Nemer et al.

2.3.3 Architecture Selection: MLP versus Attention and Graph Neural Networks

The choice of a fixed-capacity Multi-Layer Perceptron (MLP) as the policy backbone warrants explicit justification, since attention-based and graph neural network (GNN) architectures have demonstrated advantages in combinatorial routing tasks that share structural similarities with UAV coverage planning.

Kool et al. [20] show that attention mechanisms can learn permutation-invariant policies over variable-length sets of locations, a natural fit for problems where the agent must sequence visits across an unordered sensor deployment. Cappart et al. [21] demonstrate that GNN policies that explicitly represent the deployment graph can generalise across topology changes that defeat fixed-input architectures. In principle, either approach could encode inter-sensor spatial relationships that the MLP with concatenated per-sensor features cannot.

However, several properties of the primary evaluation regime favour the MLP. First, the UAV's five-action discrete space is low-dimensional; the representational bottleneck lies in the temporal protocol state, which frame stacking addresses without architectural complexity. Second, the training distribution (uniformly random sensor positions sampled each episode) does not impose consistent inter-sensor topology, so a graph-structured inductive bias offers no stable structure to exploit. The cross-layout generalisation analysis (Section 5.6) confirms this: the MLP achieves statistically significant advantages on geometrically regular layouts (grid-aligned, ring) and matches or exceeds greedy performance on dispersed layouts analogous to the training distribution. Third, MLP compatibility with the Stable-Baselines3 DQN implementation avoids the engineering overhead of custom graph-batch collation in the replay buffer, which would complicate the training pipeline without clear benefit for the primary regime.

The failure boundary is precisely identified: bimodal two-cluster deployments (-14.0%, $d = 5.33$,

Section 5.6) represent a topological regime where inter-cluster routing decisions cannot be inferred from local positional features. A curriculum fine-tuning experiment (subsubsection 5.6.3) confirms the gap is architectural rather than distributional: the MLP cannot encode cluster identity or inter-cluster routing priority regardless of training exposure. This establishes attention or GNN architectures as a motivated next step rather than a speculative one, and the performance gap quantifies the representational cost of the MLP choice in this specific failure regime.

2.3.4 Information Freshness and Domain Generalisation

Kaul et al. [22] show that the throughput-optimal update frequency is not the freshness-optimal frequency, motivating the explicit AoI term in the reward function. Abd-Elmagid et al. [23] demonstrate a fundamental AoI–throughput trade-off in IoT networks, supporting the multi-term weighted reward design adopted here.

Tobin et al. [24] introduce Domain Randomisation to bridge the sim-to-real gap, subsequently validated by OpenAI [25] at scale in robotic manipulation. Bengio et al. [26] show that curriculum learning, which presents training examples in order of increasing difficulty, produces faster convergence and better generalisation. Both techniques are combined in this work to produce a single policy deployable across a 100$\times$ variation in operational area without retraining.

2.4 Summary and Project Positioning

The literature splits along a clean fault line: path planners ignore the protocol's failure modes (creating the SF Monoculture), and protocol optimisers use static mathematics that cannot adapt to mobility. Zorbas et al. [7] name the gap directly, concluding that "reinforcement learning models could be employed to adapt over time to evolving network conditions."

2.4.1 Simulation Tool Gap

No existing open simulation framework models the complete causal chain required for protocol-aware UAV control. Bor et al.'s LoRaSim [5] provides the foundational LoRaWAN collision model but assumes a static topology and omits both ADR dynamics and the Capture Effect, making it unsuitable for mobile-gateway evaluation. Xiong et al.'s FlyingLoRa [8] introduces UAV trajectory planning over LoRa networks but omits ADR entirely (treating each sensor's data rate as fixed) and

assumes mutual packet destruction on any intra-SF collision, the exact assumption that Croce et al. [13] empirically disprove. Zorbas et al. [7] model ADR convergence accurately but use a static MILP formulation that cannot be used as a Gymnasium-compatible step environment for DRL training. No prior work therefore provides a Gymnasium-compatible environment that simultaneously models EMA-ADR convergence, the 6 dB Capture Effect threshold, Two-Ray path loss, log-normal shadowing, and asynchronous duty cycles within a single step, the combination required for an agent to learn position-induced SF-diversity exploitation.

Table 1 positions this dissertation at the specific intersection no prior work occupies: protocol-aware DRL for mobile LoRaWAN gateways with explicit Capture Effect modelling and multi-objective fairness.

Table 1. Positioning of this work relative to key related works across five dimensions. ✓ = addressed; × = not addressed.

Work	Protocol-Aware	DRL-Based	LoRaWAN Specific	Capture Effect	Multi-Objective
Zorbas et al. [7]	✓	×	✓	×	×
Xiong et al. [8]	×	×	✓	×	✓
Challita et al. [11]	✓	✓	×	×	×
Nemer et al. [15]	×	✓	×	×	✓
Luong et al. [10]	×	✓	×	×	×
This work	✓	✓	✓	✓	✓

The proposed framework treats the UAV's physical position as a direct control input for the LoRaWAN protocol, the key step that static optimisation [7] and energy-centric heuristics [8] both skip. Modelling the Capture Effect distinguishes this simulation from all prior LoRaWAN simulators that assume mutual packet destruction on any intra-SF collision.

3 Preliminaries

3.1 LoRaWAN Physical Layer

LoRa (Long Range) is a proprietary spread-spectrum modulation technique derived from Chirp Spread Spectrum (CSS). It encodes information by varying the frequency of "chirps" over time, which provides exceptional resilience to interference and noise [4], [5].

3.1.1 Spreading Factors and Orthogonality

The key parameter in LoRa modulation is the Spreading Factor (SF), which determines the duration of a chirp. LoRaWAN defines six orthogonal spreading factors, SF $\in \{7, 8, \ldots, 12\}$.

- **SF7**: Lowest range, highest data rate, shortest Time-on-Air.
- **SF12**: Longest range, lowest data rate, longest Time-on-Air.

A defining feature of LoRa is the orthogonality of these spreading factors. Signals modulated with different SFs can theoretically be received simultaneously by a gateway on the same frequency channel without interfering with each other [4], [12]. This property creates the "concurrent channel capacity" that this project aims to exploit.

3.1.2 The Capture Effect

In dense networks, collisions occur when two devices transmit on the same SF and frequency simultaneously. However, LoRa receivers exhibit the Capture Effect: if one signal is sufficiently stronger than the interfering signal (typically by a margin of 6 dB), the receiver can successfully decode the stronger packet while suppressing the weaker one [7], [13]. This physical phenomenon is modelled in the simulation to resolve co-SF collisions.

3.2 Channel Propagation Model

To accurately simulate the signal strength (RSSI) received by the UAV, a simple Free Space Path Loss (FSPL) model is insufficient. This project employs the Two-Ray Ground Reflection Model [27], which

accounts for both the direct Line-of-Sight (LoS) path and the path reflected off the ground. The ground-reflection geometry is non-negligible for low-altitude UAV platforms [28].

The received power P_r at distance d is approximated by:

$$P_r = P_t G_t G_r \frac{h_t^2 h_r^2}{d^4} \tag{1}$$

where P_t is the transmit power, G_t and G_r are antenna gains, and h_t and h_r are the heights of the transmitter (sensor) and receiver (UAV). Unlike the $1/d^2$ decay of free space, the Two-Ray model predicts a steeper $1/d^4$ decay at long distances, providing a more realistic constraint for the UAV's data collection radius.

3.3 Deep Reinforcement Learning

3.3.1 Markov Decision Process

Reinforcement Learning (RL) formalises sequential decision-making as a Markov Decision Process (MDP) [10], defined by the tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$, where $\mathcal{S}$ is the finite set of states, $\mathcal{A}$ is the finite set of actions, $P(s'|s,a)$ is the transition probability of moving to state s' from state s after executing action a, $R(s,a)$ is the immediate reward, and $\gamma \in [0,1]$ is the discount factor that weights future rewards relative to immediate ones.

At each timestep t, the agent observes the current state $s_t \in \mathcal{S}$, selects an action $a_t \in \mathcal{A}$ according to a policy $\pi : \mathcal{S} \to \mathcal{A}$, receives reward $r_t = R(s_t, a_t)$, and transitions to s_{t+1}. The goal is to find the optimal policy π^* that maximises the expected discounted cumulative reward:

$$\pi^* = \arg\max_{\pi} \ \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t)\right] \tag{2}$$

The state-value function $\mathcal{V}^\pi(s)$ quantifies the long-run return obtained by following policy π from state s:

$$\mathcal{V}^\pi(s) = \mathbb{E}_\pi\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \;\middle|\; s_0 = s\right] \tag{3}$$

The optimal value function satisfies the Bellman optimality equation $\mathcal{V}^*(s) = \max_a \{\mathbb{E}\left[R(s,a) + \gamma \mathcal{V}^*(s')\right]\}$. Defining the optimal action-value (Q) function as

$Q^*(s,a) \triangleq R(s,a) + \gamma\,\mathbb{E}[\mathcal{V}^*(s')]$, the optimal policy reduces to greedy selection: $\pi^*(s) = \arg\max_a Q^*(s,a)$.

3.3.2 Partially Observable Markov Decision Process

In many real-world problems the agent cannot observe the full system state; it receives only a noisy or incomplete observation. Such settings are modelled as Partially Observable Markov Decision Processes (POMDPs) [10], defined by the 6-tuple $(\mathcal{S}, \mathcal{A}, P, R, \Omega, \mathcal{O})$, where $\mathcal{S}$, $\mathcal{A}$, P, and R retain their MDP meanings, Ω is the set of possible observations, and $\mathcal{O}(o \mid s, a, s')$ is the probability of receiving observation $o \in \Omega$ after transitioning from s to s' via action a.

At each step the agent does not see s directly; it receives $o_t \sim \mathcal{O}(\cdot \mid s_{t-1}, a_{t-1}, s_t)$ and must act on this partial information. Despite this, the agent's goal remains the same: find the policy π^* that maximises the expected long-term discounted reward $\sum_{t=0}^{\infty} \gamma^t r(s_t, \pi^*(o_t))$.

The UAV data-collection task in this work is inherently a POMDP: the agent observes its own position and battery level exactly, but sensor buffer states and LoRa channel conditions for out-of-range nodes are hidden. Frame stacking (Section **??**) is used to approximate the belief state by aggregating the last $k = 4$ observations, restoring an approximate Markov property and enabling standard DQN training on what is formally a partially observable problem.

Figure 1 illustrates the three core components of the DRL framework used in this work: the RL interaction loop, the DQN network architecture, and the full Deep Q-Learning system with experience replay.

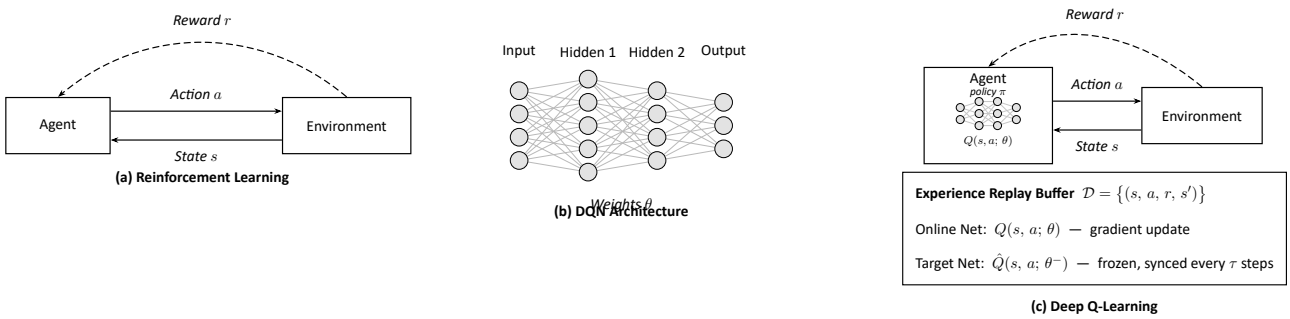


Fig. 1. Deep reinforcement learning framework. (a) The RL interaction loop: the DQN agent selects actions (N/S/E/W/Hover) based on the current state and receives a reward signal encoding data throughput, fairness, and energy consumption. (b) The DQN network architecture with fully connected layers [512, 512, 256] and ReLU activations, mapping a 1530-dimensional frame-stacked state to Q-values for five actions. (c) The complete Deep Q-Learning system, combining the RL loop with experience replay buffer and a frozen target network $\hat{Q}(s, a; \theta^-)$ for training stability.

3.3.3 Deep Q-Networks (DQN)

Standard Q-Learning is computationally intractable for high-dimensional state spaces. Mnih et al. [16] introduced the Deep Q-Network (DQN), which uses a neural network to approximate $Q(s, a; \theta)$ by minimising the temporal difference error:

$$L(\theta) = \mathbb{E}\left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta)\right)^2\right] \tag{4}$$

Two mechanisms stabilise training: **experience replay** breaks temporal correlation between consecutive observations by sampling uniformly from a fixed-capacity transition buffer; **target network stabilisation** maintains a periodically frozen copy $\hat{Q}(\cdot; \theta^-)$ to prevent the Q-value feedback loop that causes divergence. Both are essential here because EMA-ADR convergence causes the same state-action pair to yield different rewards at different training stages.

4 Methodology

This chapter details the development of the protocol-aware Deep Reinforcement Learning (DRL) framework. We first present the system model, which mathematically defines the UAV's kinematics, energy consumption, and the stochastic LoRaWAN channel. Subsequently, we formulate the data collection task as a Markov Decision Process (MDP), defining the specific state space, action space, and multi-objective reward function used to train the agent. Finally, we describe the algorithmic implementation of the Deep Q-Network (DQN) within the custom Gymnasium simulation environment.

4.1 System Model

We consider a remote IoT monitoring environment comprising a set $\mathcal{N} = \{1, \ldots, N\}$ of static sensor devices uniformly distributed over a rectangular Region of Interest (RoI) $\mathcal{W} \subset \mathbb{R}^2$. These devices continuously generate monitoring data, which accumulates in finite transmission buffers. Unlike simple deadline-based models, we assume data carries value only if it is collected before the buffer capacity B_{max} is exceeded, after which packet loss occurs.

A rotary-wing UAV is the mobile LoRaWAN gateway, dispatched to harvest data from the ground sensors. The mission duration is fixed to a finite horizon T and divided into discrete time steps

$t = 0, 1, \ldots, T$, each of duration Δt. The UAV flies at fixed altitude h to maintain a consistent ground-to-air channel geometry, updating its horizontal position $p_t = (x_t, y_t)$ at each step. The system employs the LoRaWAN protocol with orthogonal Spreading Factors (SF), enabling simultaneous uploads from multiple sensors when they are assigned distinct SFs by the Adaptive Data Rate (ADR) mechanism. The UAV must therefore optimise its trajectory to actively manage spectral assignments and prevent network congestion.

Assumption 1. *The UAV operates at fixed altitude h, reducing trajectory optimisation to a 2D horizontal problem; vertical dynamics are abstracted into fixed power profiles P_{hover} and P_{flight}.*

Assumption 2. *The LoRaWAN gateway is single-channel multi-SF: at most one transmission per SF is decoded simultaneously; intra-SF collisions are resolved via the Capture Effect (Section 4.1.4).*

Assumption 3. *Sensors are LoRaWAN Class A devices with a 10% duty cycle*[1] *and Poisson data generation; network-initiated wake-up is infeasible, so the UAV must arrive during a node's natural transmission window.*

Assumption 4. *The UAV retains a 20% battery reserve ($0.20 \times E_{\text{max}} = 54.8$ Wh) for safe return-to-base. The episode terminates when remaining energy reaches this threshold. Upon landing, a fully charged relay UAV is immediately dispatched to continue the mission; continuous area coverage is therefore maintained by a two-UAV rotation, and each agent is evaluated on its performance within a single flight.*

The relay-handover model is operationally realistic: commercial LoRaWAN UAV deployments (e.g. DJI TB60 with a charging dock) use automatic battery-swap or relay dispatch to sustain continuous data harvesting over long missions [2]. The effective episode energy budget is therefore $E_{\text{eff}} = 0.80 \times 274 = 219.2$ Wh, reducing the maximum flight steps proportionally. This tightens the feasibility constraints derived in Section 4.1.6: the maximum traversable distance in a single episode becomes $d_{\text{max}} \approx 15{,}782$ m, making full coverage at $1{,}000 \times 1{,}000$ scale even more infeasible (TSP tour $\approx 44{,}700$ m, a factor of 2.8 beyond d_{max}).

[1] The EU 868 MHz ISM band mandates a 1% radio duty cycle. The 10% parameter models the *effective* per-timestep transmission probability (wake-up rate $\times$ packet frequency $\times$ channel access). Sensitivity is tested down to 4% in Section 5.6.

4.1.1 UAV Kinematics and Energy Consumption

We consider a rotary-wing Unmanned Aerial Vehicle (UAV) operating within a discretised 3D grid space $\mathcal{W} \subset \mathbb{R}^3$. The mission is defined over a finite time horizon T, discretised into time steps of duration Δt. At each time step k, the UAV's kinematic state is defined by its 3D position $\mathbf{p}_k = [x_k, y_k, z_k]^T$.

The UAV is controlled by a discrete action space $\mathcal{A}$ consisting of five high-level motion primitives:

$$\mathcal{A} = \{\text{MoveN},\ \text{MoveS},\ \text{MoveE},\ \text{MoveW},\ \text{Hover}\} \tag{5}$$

Each movement action updates the UAV's position by a fixed step size δ_d in the corresponding cardinal direction. The Hover action maintains the current position $\mathbf{p}_{k+1} = \mathbf{p}_k$, which is critical for stabilising the LoRaWAN link to facilitate data packet reception.

The energy consumption model is state-dependent. Let E_k denote the remaining battery energy at step k. The energy evolution is governed by:

$$E_{k+1} = E_k - P(a_k) \cdot \Delta t \tag{6}$$

where $P(a_k)$ is the instantaneous power consumption. We define distinct power profiles for flight modes:

$$P(a_k) = \begin{cases} P_{\text{hover}} & \text{if } a_k = \text{Hover} \\ P_{\text{flight}} & \text{if } a_k \in \{\text{Move}\} \end{cases} \tag{7}$$

Consistent with rotary-wing aerodynamics, we model the power consumption such that $P_{\text{hover}} > P_{\text{flight}}$ due to the lack of translational lift in static flight.[2] Accordingly, we assign a higher power cost to the Hover action (700 W) compared to translational Move actions (500 W). This power differential incentivises the agent to optimise for efficient flight trajectories and penalises excessive loitering, thereby aligning the reward signal with the physical reality of limited onboard energy storage.

[2] In rotary-wing UAVs, translational lift (the additional lift gained from forward flight) means that hovering requires more power than forward flight at moderate speeds.

4.1.2 Channel Model and Protocol Modelling

The ground-to-air channel is modelled using the Two-Ray Ground Reflection model [27], which is appropriate for low-altitude UAV platforms where the direct LoS path and ground-reflected path interfere. The received power is:

$$P_r = P_t G_t G_r \frac{h_t^2 h_r^2}{d^4} \tag{8}$$

where h_t, h_r are transmitter and receiver heights, d is the 3D slant range, and G_t, G_r are antenna gains. The quartic decay ($1/d^4$) enforces realistic data-collection radius constraints and means small changes in the UAV's position have a magnified impact on RSSI, a property that the DRL agent must learn to exploit [28].

4.1.3 Device Class and Traffic Model

Sensor nodes are modelled as LoRaWAN Class A devices [4], operating asynchronously with an effective 10% per-timestep transmission probability (see Assumption 3 footnote). Data generation follows a Poisson process with rate λ_i per sensor, accumulating in finite buffers of capacity B_{max}. Once a buffer is full, new packets are discarded, creating a hard collection deadline for the agent. This models realistic energy-harvesting deployments where network-initiated wake-up is infeasible.

Propagation delay and ADR convergence latency. Radio propagation delay over the maximum collection range of 3,650 m is $\tau_{\text{prop}} = d/c \approx 12\,\mu$s, six orders of magnitude below the 1-second timestep duration and therefore not modelled. The dominant latency in the system is instead *ADR convergence*: when the UAV repositions to reduce path loss, the EMA filter ($\hat{\text{RSSI}}_{t+1} = (1-\lambda)\hat{\text{RSSI}}_t + \lambda\,\text{RSSI}_t$, $\lambda = 0.1$) requires approximately $1/\lambda = 10$ timesteps ($\sim$10 s) before the SF assignment falls to the lower, higher-throughput value. This inertia is intentional: it prevents SF from oscillating due to transient shadowing and forces the agent to commit to a position long enough for ADR to converge before flying elsewhere.

4.1.4 Collision Resolution via Capture Effect

Intra-SF collisions (multiple sensors on the same spreading factor) are resolved using the Capture Effect [13]: the gateway decodes sensor i if its received power exceeds the interference from all

co-channel sensors by the capture threshold $\tau_{\text{cap}} = 6$ dB:

$$\frac{P_{r,i}}{\sum_{j \in \mathcal{I}_{\text{co-SF}}} P_{r,j} + N_0} \geq \tau_{\text{cap}} \tag{9}$$

This forces the agent to optimise its trajectory not merely for minimum RSSI coverage but to induce the power differential required for successful capture during congestion events, a spatial optimisation that greedy heuristics cannot plan for. For two sensors colliding on the same SF, the capture condition simplifies to $d_{j,t}^4 / d_{i,t}^4 \geq \tau_{\text{cap}}$, requiring the UAV to position itself strictly closer to the desired sensor.

Timestep-level collision model and SF independence. A key question is whether an ongoing SF reception can be interrupted by a later-arriving transmission on the same SF. In real LoRa hardware, a late-arriving co-SF signal can corrupt an already-locked packet if it exceeds the noise floor; however, transmissions on *different* SFs are fully orthogonal and never interrupt one another.

The simulation resolves collisions at the *timestep* level rather than the packet-timing level, which is standard practice in LoRaWAN system simulation [4], [13]. At each collect step the procedure is:

1. Every in-range sensor with buffered data independently decides to transmit with probability $P_{\text{overall}} = P_{\text{link}} \times P_{\text{cycle}}$, where $P_{\text{link}} = \sigma(\text{SNR}_{\text{dB}} - \text{SNR}_{\text{req}}(\text{SF}))$ is a sigmoid link-margin probability and $P_{\text{cycle}} = 0.10$ is the duty-cycle probability.

2. Sensors are grouped by their current SF. Each SF group is an independent channel, transmissions across different SF groups never collide.

3. Within each SF group, if exactly one sensor transmits: it is decoded unconditionally. If two or more transmit simultaneously: the Capture Effect criterion (Eq. (9)) is applied. If the strongest signal exceeds all interference by $\tau_{\text{cap}} = 6$ dB, it is decoded; otherwise destructive interference destroys all packets in that group.

4. Data is collected from every SF group that produced a winner, up to one decoded packet per SF per timestep.

This four-phase model means the maximum concurrent throughput per timestep is $|\mathcal{S}| = 4$ packets (one per SF in $\{7, 9, 11, 12\}$), achieved only when the UAV has positioned itself such that sensors are distributed across all four SFs , precisely the condition the DQN learns to engineer.

4.1.5 Gateway Architecture

The UAV acts as a single-channel, multi-SF LoRaWAN gateway. It is important to be precise about what this means, as it is central to both the problem and the DQN's learned strategy.

One channel, one device per SF, one transmission at a time per SF. The gateway operates on a single radio frequency channel. Within that channel, LoRa Spreading Factors are *quasi-orthogonal*: transmissions on different SFs occupy the same bandwidth but are distinguishable by the gateway's correlator bank because each SF uses a different chirp rate [4]. Consequently, the gateway can decode *one uplink packet per SF simultaneously*, up to four concurrent receptions across $\mathcal{S} = \{7, 9, 11, 12\}$. This is not multi-channel reception; it is the inherent spectral orthogonality of LoRa modulation exploited by a single radio.

Class A device constraint. All sensors are LoRaWAN Class A devices [4]. Class A is the most restrictive device class: a sensor opens its two downlink receive windows only *immediately after* completing a self-initiated uplink. At all other times the radio is off. This has two direct consequences for the UAV mission:

1. **No network-initiated wake-up.** The UAV gateway cannot command a sensor to transmit. It must be within collection range *at the moment* a sensor naturally opens its uplink window. The 10% effective duty-cycle probability models this constraint: at each timestep, each in-range sensor independently transmits with probability $P_{\text{cycle}} = 0.10$.

2. **Uplink-limited throughput.** Data rate is bounded by the sensor's SF and duty-cycle window. A sensor on SF12 (250 bits/s effective) transfers far fewer bytes per window than a sensor on SF7 (5470 bits/s effective), so inducing ADR convergence to lower SFs directly increases bytes collected per visit.

Why SF diversity matters. Because the gateway decodes one device per SF, having K sensors within range on K *distinct* SFs yields K simultaneous decodable transmissions per step. If all K sensors share the same SF (SF Monoculture), at most one is decoded (via Capture Effect) and the remaining $K - 1$ transmissions are lost. The UAV's positioning therefore controls not just link quality (RSSI $\rightarrow$ SF assignment via ADR) but the number of concurrent receptions, making

trajectory planning a direct lever on network throughput. This is the mechanism the DQN learns to exploit and that one-step greedy heuristics cannot plan for.

4.1.6 Link Budget, Collection Radius, and Operational Envelope

A central design choice is the UAV operating altitude $h_r = 100$ m. This is the regulatory ceiling for uncontrolled airspace operations under EU and UK drone legislation (the "open category"), allowing deployment without special authorisation. The DJI TB60, on which the energy model is based, is rated for operations up to 120 m AGL. At 100 m altitude the Two-Ray model applies: the direct LoS path and ground-reflected path interfere constructively, producing the $1/d^4$ decay of Eq. ((8)). Each grid unit represents 10 m of ground distance.

Deriving collection radii. Substituting the simulation parameters ($P_t = 14$ dBm, $G_t = G_r = 0$ dBi, $h_t = 1.5$ m, $h_r = 100$ m) into the Two-Ray received-power equation and inverting for horizontal range r (the ground-projected distance from the sensor to the UAV ground-track) yields:

$$r = \left(10^{\frac{P_t + 20\log_{10}(h_t h_r) - \text{RSSI}_{\text{min}}}{40}}\right) \tag{10}$$

The ADR mechanism maps RSSI to Spreading Factor; the gateway can only decode a transmission if RSSI exceeds the SF-specific demodulation floor (Table 2).

Table 2. Spreading Factor assignment thresholds and maximum collection radii derived from Eq. ((10)). Sensors beyond $r_{\text{max}} = 365$ grid units are out of collection range.

SF	RSSI threshold (dBm)	Range (m)	Range (grid units)
SF7	-60	860	86
SF9	-70	1,540	154
SF11	-78	2,440	244
SF12	-85	3,650	365
Out of range	<-85	,	>365

Justification of evaluation grid scales. The grid scales are chosen to span qualitatively distinct regimes defined by the interaction between $r_{\text{max}} = 365$ grid units and the grid geometry (Table 3).

At 100×100 the maximum corner-to-corner distance (141 units) is smaller than r_{max} (365 units), so a sensor is always within collection range regardless of UAV position: navigation has no link-budget effect and the problem reduces to SF diversity management alone. At 500×500 the diagonal (707

Table 3. Operational regime for each evaluation grid. TSP tour estimate uses $\bar{d} \approx \sqrt{L^2/N}$ as the mean inter-sensor hop distance for $N = 20$ sensors on an $L \times L$ grid.

Grid	Physical size	Max corner dist.	Est. TSP tour	Regime
100×100	$1\,\text{km}^2$	141 units	4,5 km	All sensors always in range
300×300	$9\,\text{km}^2$	424 units	13,4 km	Navigation matters; coverage feasible
500×500	$25\,\text{km}^2$	707 units	22,4 km	Link budget active; marginal feasibility
1000×1000	$100\,\text{km}^2$	1,414 units	44,7 km	Physically infeasible for single UAV

units) is nearly double r_{max}, meaning sensors near opposite corners are mutually unreachable without navigation.

The energy-feasibility bound is derived from the battery model. With $E_{\text{max}} = 274$ Wh and flight power $P_{\text{flight}} = 500$ W per step ($\Delta t = 1$ s per step), the maximum traversable distance is:

$$d_{\text{max}} = \frac{E_{\text{max}}}{P_{\text{flight}} \cdot \Delta t} \times \delta_d = \frac{274\,\text{Wh} \times 3600\,\text{s/h}}{500\,\text{W} \times 1\,\text{s}} \times 10\,\text{m} = 19{,}728\,\text{m} \tag{11}$$

For $N = 20$ sensors on a $1{,}000 \times 1{,}000$ grid, the estimated TSP tour length of 44,700 m exceeds $d_{\text{max}} = 19{,}728$ m by a factor of 2.3: no agent, regardless of policy quality, can visit all sensors within the energy budget. This establishes the $1{,}000 \times 1{,}000$ grid as a *physically infeasible* condition and the 500×500 grid as the upper boundary of the single-UAV operational envelope. These four grid scales therefore systematically probe from trivially solvable to physically impossible, enabling a principled characterisation of where protocol-aware navigation adds value.

4.2 Problem Formulation

We formulate the UAV trajectory optimisation problem as a finite-horizon Partially Observable Markov Decision Process (POMDP). The agent does not observe sensor locations directly, and the link-quality signal $Q_{i,t}$ is zero for all out-of-range sensors, leaving channel state and SF distributions beyond the UAV's current footprint hidden. The objective is to maximise total collected data throughput subject to energy, kinematic, and protocol constraints:

$$\text{(P1):} \quad \max_{\pi} \quad \sum_{t=0}^{T} \sum_{i=1}^{N} D_{i,t} \tag{12}$$

$$\text{s.t.} \quad \text{RSSI}_{i,t}(p_t) \geq \text{RSSI}_{\text{min}}(\text{SF}_{i,t}) \quad \forall i \in \mathcal{N} \tag{13}$$

$$E_{t+1} = E_t - P(a_t) \cdot \Delta t \geq 0 \quad \forall t \tag{14}$$

$$\sum_{i \in \mathcal{N}} \mathbb{I}(\text{SF}_{i,t} = k) \leq 1 \quad \forall k \in \mathcal{S} \tag{15}$$

where $D_{i,t}$ is the data successfully collected from sensor i at step t, E_t is the remaining battery energy, and constraint ((15)) enforces SF orthogonality (one active transmission per SF).

Proposition 1 (Intractability and Markov Restoration). *Under Assumptions 1–3, (P1) is non-convex: p_t and* $\text{SF}_{i,t}$ *are coupled through the EMA-ADR filter (λ=0.1), violating the Markov property. Classical methods (MILP, convex MPC) cannot tractably encode this history. Frame stacking* $\mathbf{S}_t^{\text{stacked}}$ *(Eq. (19)) restores approximate Markovianity by rendering EMA-ADR convergence state-observable.*

4.2.1 State Space Definition

The observation at step t is:

$$\mathbf{S}_t = [p_t,\ E_t,\ B_t,\ U_t] \tag{16}$$

where $p_t \in \mathbb{R}^2$ is the UAV's 2D position (altitude is fixed at h_u), $E_t \in [0, E_{\text{max}}]$ is remaining battery energy, $B_t \in [0, B_{\text{max}}]^N$ is the per-sensor buffer occupancy, and $U_t \in [0, 1]^N$ is a normalised urgency vector:

$$U_{i,t} = \frac{B_{i,t}}{B_{\text{max}}} \cdot \frac{\text{data_generation_rate}_i}{\text{collection_rate}_i} \tag{17}$$

Urgency captures imminent overflow risk more directly than classical Age of Information [22]: a sensor with a near-full buffer and a fast generation rate has high urgency regardless of how long its data has been waiting. A third per-sensor feature, link quality $Q_{i,t} \in [0, 1]$, encodes the sensor's current Spreading Factor normalised to $[0, 1]$ (SF7 $\mapsto$ 1.0, SF12 $\mapsto$ 0.1), providing the agent with a direct signal of spectral efficiency at each sensor:

$$Q_{i,t} = \begin{cases} \text{SF_norm}(\text{SF}_{i,t}) & \text{if sensor } i \text{ in range} \\ 0 & \text{otherwise} \end{cases} \tag{18}$$

The total state dimensionality is therefore $3 + 3N$ (UAV position, battery, and per-sensor buffer, urgency, and link quality). The link quality feature allows the agent to observe ADR convergence progress directly, complementing the temporal trend information provided by frame stacking.

To make the environment approximately Markovian under partial observability, we employ frame stacking with $k = 4$ steps:

$$\mathbf{S}_t^{\text{stacked}} = [\mathbf{S}_{t-3},\ \mathbf{S}_{t-2},\ \mathbf{S}_{t-1},\ \mathbf{S}_t] \in \mathbb{R}^{4(3+3N)} \tag{19}$$

The temporal window lets the agent observe urgency and link-quality trends, inferring SF

convergence dynamics (link quality rising over consecutive steps signals EMA-ADR downgrading from SF12 toward SF7) without explicitly tracking the ADR filter state.

4.2.2 Action Space Definition

The discrete action space $\mathcal{A}$ (Eq. (5)) is as defined in Section 4.1.1. Hover is energetically more expensive per unit displacement ($P_{\text{hover}} > P_{\text{flight}}$) but essential for stabilising the LoRa link during data collection and for allowing duty-cycle windows to open.

4.3 Sensor Placement in Simulation

Figure 2 illustrates the complete system model for the 500×500 evaluation condition. The UAV (DJI TB60) flies at 100 m altitude and the concentric rings show the SF-specific collection radii derived in Section 4.1.6. The charging station marks the return-to-base point (Assumption 4); a relay UAV is dispatched upon landing to maintain continuous coverage.

4.3.1 Stochastic Sensor Deployment

Sensor coordinates are sampled from $\mathcal{U}(0, L) \times \mathcal{U}(0, W)$, modelling an unstructured ad-hoc topology. This ensures the agent learns a generalisable policy rather than memorising a fixed map, while also naturally creating density clusters that induce the SF Monoculture.

4.3.2 Multi-Objective Reward Function

A dense reward is used to avoid the vanishing gradient problem inherent in sparse mission-success objectives. The total reward at step t is:

$$R_t = w_{\text{coll}} \cdot R_{\text{coll}} - w_{\text{energy}} \cdot C_{\text{energy}} - w_{\text{AoI}} \cdot C_{\text{AoI}} - w_{\text{hover}} \cdot C_{\text{hover}} \tag{20}$$

Data collection (R_{coll}): positive reward for every successfully decoded packet, encouraging positioning that exploits Capture Effect and orthogonal-SF concurrent reception.

Energy penalty ($C_{\text{energy}} = P(a_t) \cdot \Delta t$): proportional to action power draw, discouraging meandering trajectories.

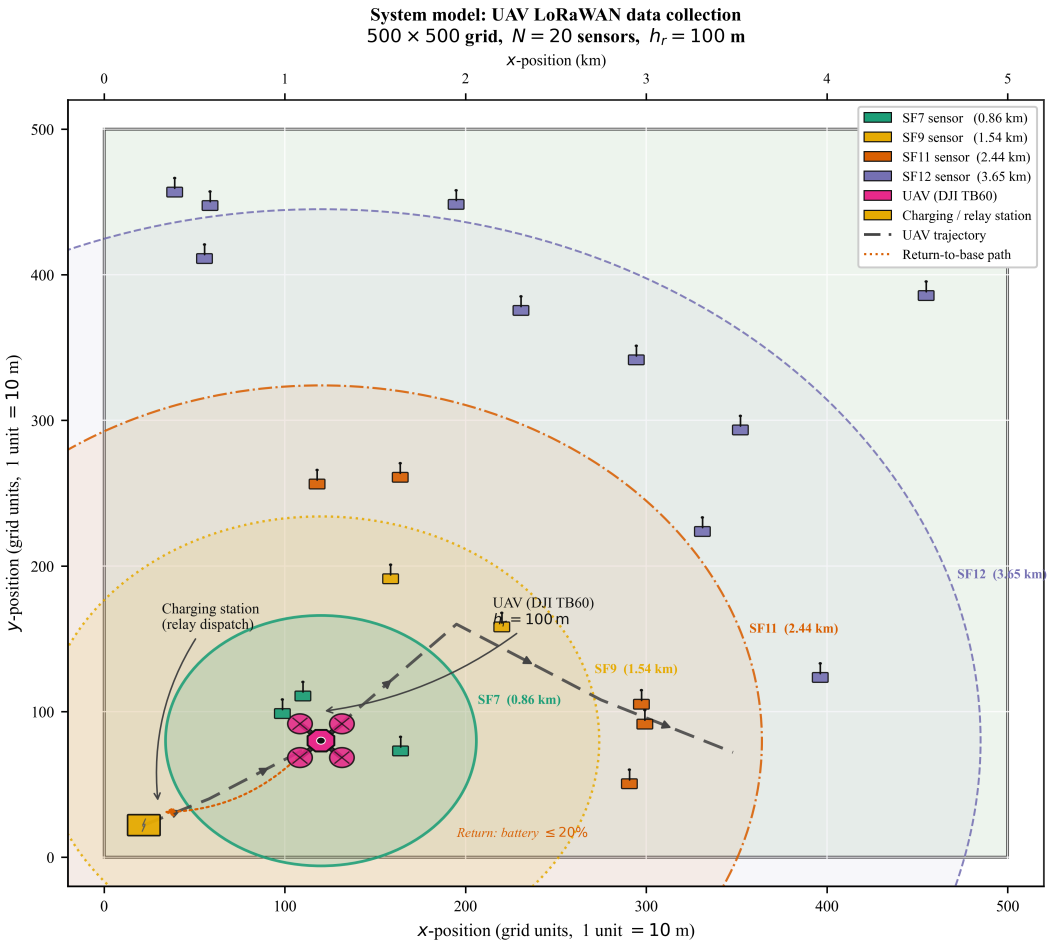


Fig. 2. System model for the 500×500 evaluation condition ($N = 20$ sensors, $h_r = 100$ m). Sensor markers are coloured by their current Spreading Factor assignment. Concentric rings show the link-budget collection radius for each SF (Table 2). The dashed arrow indicates the UAV trajectory; the dotted arrow the return-to-base path on reaching the 20% battery reserve threshold. A relay UAV is dispatched from the charging station to continue the mission.

AoI penalty ($C_{\text{AoI}} = \frac{1}{N}\sum_i \text{AoI}_{i,t}$ [22], [23]): mean Age of Information across all sensors, forcing the agent to service stale nodes and preventing information starvation.

Hover penalty ($C_{\text{hover}} = \mathbb{I}(a_t = \text{HOVER}) \cdot \lambda_{\text{hover}}$): a small fixed cost applied to all action types (including movement) to prevent the agent from idling indefinitely over a single high-rate sensor, a failure mode observed in earlier reward formulations.

This shaped structure provides immediate step-level feedback that guides the agent toward the long-horizon multi-objective optimum.

Reward scale rationale. Component magnitudes are calibrated so that visiting all sensors is strictly preferable to revisiting a single node ($+5{,}000$ discovery vs. $-2{,}000$ terminal penalty per unvisited sensor), and no component exceeds 15% of total episode reward.

4.4 Deep Reinforcement Learning Implementation

4.4.1 Deep Q-Network (DQN) Architecture

The agent's decision-making policy is approximated using a Deep Q-Network (DQN) [16]. We employ a Multi-Layer Perceptron (MLP) architecture to map the high-dimensional state vector $\mathbf{S}_t$ to Q-values for the five discrete actions.

The network is a 3-layer MLP with hidden dimensions [512, 512, 256] and ReLU activations. The input dimension is $4(3 + 3N)$ (frame-stacked state, Section 4.2.1) and the output is 5 Q-values, one per action in $\mathcal{A}$.

The network weights θ are optimised using the Adam optimiser to minimise the temporal difference error between the predicted Q-values and the target Q-values derived from the Bellman equation. To ensure training stability, a separate target network $\hat{Q}$ with parameters θ^- is maintained and updated periodically (every C steps).

4.4.2 Probabilistic Link-Layer Model

Packet reception is modelled as a stochastic Bernoulli trial rather than a binary range predicate. The link success probability is a sigmoid function of the received SNR margin:

$$P_{\text{link}} = \sigma(\text{SNR}_{\text{dB}} - \text{SNR}_{\text{req}}(\text{SF})) = \frac{1}{1 + \exp\big(-(\text{SNR}_{\text{dB}} - \text{SNR}_{\text{req}}(\text{SF}))\big)} \tag{21}$$

where $\text{SNR}_{\text{dB}} = \text{RSSI} - N_0$ is the instantaneous signal-to-noise ratio derived from the Two-Ray path-loss model, and $\text{SNR}_{\text{req}}(\text{SF})$ is the SF-specific demodulation threshold [4]. The overall per-step transmission probability combines the link margin with the protocol duty-cycle constraint:

$$P_{\text{overall}} = P_{\text{link}} \cdot P_{\text{cycle}} \tag{22}$$

A transmission is deemed successful if $P_{\text{overall}} > u$, where $u \sim \mathcal{U}(0, 1)$ is drawn independently at each step. This formulation ensures that collection yield depends continuously on SNR margin: sensors at the edge of the UAV's footprint contribute probabilistically rather than being admitted or excluded by a hard range boundary, coupling physical positioning directly to link-layer throughput.

4.4.3 Training Convergence

The DQN was trained across three curriculum stages progressively introducing all 16 training conditions (4 grid sizes $\times$ 4 sensor counts). Within each stage, the mean episode reward follows a characteristic rise-and-plateau trajectory: early episodes produce large negative rewards as the agent explores randomly, followed by a monotonic increase as the policy learns to collect data, converging to a stable plateau. Later stages converge to progressively higher absolute rewards (3–5×10^6), confirming that the curriculum successfully bootstraps the agent from easy to hard conditions.

The TD loss exhibits a spike-then-converge profile at each stage transition: the spike reflects the temporary distribution shift as new grid sizes and sensor counts enter the training mix, while convergence confirms the replay buffer and target network stabilise learning despite this non-stationarity. No divergence occurs across any stage transition, validating the curriculum ordering (small grids with few sensors first, progressing to large grids with dense deployments). Full convergence curves for both reward and TD loss across all stages are provided in Appendix I (Figure 7).

4.5 Architecture and Interface

The simulation is architected as a custom environment compliant with the Gymnasium (formerly OpenAI Gym) interface [29], enabling integration with standard DRL libraries such as Stable-Baselines3 [30]. The environment encapsulates the complex cyber-physical interactions into a discrete time-stepping logic.

Each `step(action)` call sequentially: (1) updates UAV position and decrements battery; (2) recomputes RSSI via the Two-Ray model; (3) updates AoI counters, generates Poisson packets, and advances EMA-ADR; (4) resolves collisions via Capture Effect; and (5) computes the multi-objective reward R_t.

4.5.1 Training Algorithm

At each step the agent selects $a_t = \arg\max_a Q(s_t, a; \theta)$ (or a random action with probability ϵ), executes the Gymnasium step, stores the transition (s_t, a_t, r_t, s_{t+1}) in the replay buffer $\mathcal{D}$, and

updates θ by minimising the TD loss on a sampled minibatch. The target network $\hat{Q}$ is hard updated every C steps to stabilise training. The complete pseudocode is provided in Appendix G.

4.5.2 Hyperparameters

The DQN was trained for 2×10^6 timesteps, learning rate $\alpha = 3 \times 10^{-4}$, discount $\gamma = 0.99$, replay buffer of 150,000 transitions, and batch size 128. A target network is updated every 1,000 steps. ϵ-greedy exploration decays from 1.0 to 0.05 over the first 25% of training steps. The policy network is a 3-layer MLP with hidden dimensions [512, 512, 256] and ReLU activations. Full hyperparameter listings are provided in Appendix G.

4.6 Summary

This chapter presented the protocol-aware DRL framework: the system model (UAV kinematics, LoRaWAN channel, Capture Effect), the MDP formulation with dense multi-objective reward, and the DQN training pipeline. The following chapter evaluates the agent against greedy baselines.

5 Results and Discussion

This chapter evaluates the trained DQN agent against two greedy baselines across network densities ($N \in \{10, 20, 30, 40\}$) and grid sizes (100×100 to 1,000×1,000). All comparisons use 20–30 seeds with Welch's t-test and Cohen's d. Sections cover experimental setup, trajectory analysis, multi-seed performance, ablations (Section 5.5), and a discussion of mechanisms and limitations (Section 5.7).

5.1 Experimental Setup

All agents are evaluated on the high-fidelity Gymnasium simulation with the UAV flying at a fixed altitude of 100 m and each grid unit equal to 10 m. The battery capacity is 274 Wh (modelled on the DJI TB60) and the episode horizon is 2,100 timesteps. The DQN policy is deterministic at test time ($\varepsilon = 0$) and uses the zero-padded observation vector ($N_{\text{max}} = 50$) as during training, with frame stacking $k = 4$.

Evaluation Sweeps. Sweep A varies sensor count $N \in \{10, 20, 30, 40\}$ on a fixed 500×500 grid. Sweep B varies grid size $\{100 \times 100,\ 300 \times 300,\ 500 \times 500,\ 1{,}000 \times 1{,}000\}$ with fixed $N = 20$. Each condition uses twenty independent random seeds; all three agents are compared on identical sensor layouts.

Baseline Agents. **Nearest Greedy** moves to the closest sensor with buffered data at each step. **SF-Aware Greedy** scores candidates as $\text{Score}_i = (13 - \text{SF}_i) \cdot w_1 - d_i \cdot w_2$, prioritising SF7–SF9 sensors for higher instantaneous throughput. A random-walk agent was not included as a formal baseline because both greedy heuristics already achieve 100% sensor coverage and competitive throughput; the relevant comparison is whether the DQN's learned policy outperforms *competent* planning strategies, not whether it outperforms undirected exploration.

Evaluation Metrics. Primary metrics are cumulative reward, network coverage (fraction of sensors visited), Jain's Fairness Index [31]:

$$\mathcal{J} = \frac{\left(\sum_i r_i\right)^2}{N \sum_i r_i^2} \tag{23}$$

and energy efficiency (bytes/Wh). Statistical significance is assessed via Welch's t-test (two-sided, $\alpha = 0.05$) on the twenty-seed reward distributions.

5.2 Single-Episode Trajectory Analysis

The trained DQN agent achieves 100% sensor coverage by step 200 on the 20-sensor, 500×500 condition (mean ±1 std across $n = 20$ seeds; see Appendix I, Figure 8). The agent sustains data collection until battery depletion at approximately step 1,400; the representative seed (seed 42) reaches depletion at step 1,404, collecting 57,335 bytes at a mean efficiency of 213.6 B/Wh. Cumulative reward grows monotonically without plateauing, indicating that the agent does not idle once early sensors are visited, a direct consequence of the buffer-overflow penalty encouraging continued collection from revisited sensors.

5.2.1 Spatial Trajectory Comparison

Figure 3 shows the UAV flight paths for all three agents on the same sensor layout (seed 42, $N = 20$), with sensor positions coloured by their initial SF assignment. The contrast is qualitatively striking.

Both greedy agents execute near-straight sweeps from sensor to sensor (100% hover fraction); the **SF-Aware** variant prioritises SF7–SF9 sensors first. The **DQN** agent, by contrast, devotes 2.9% of steps to deliberate movement ($p = 0.002$, Welch's t-test, 10 seeds), navigating towards positions that simultaneously place multiple sensors within range across orthogonal SFs.

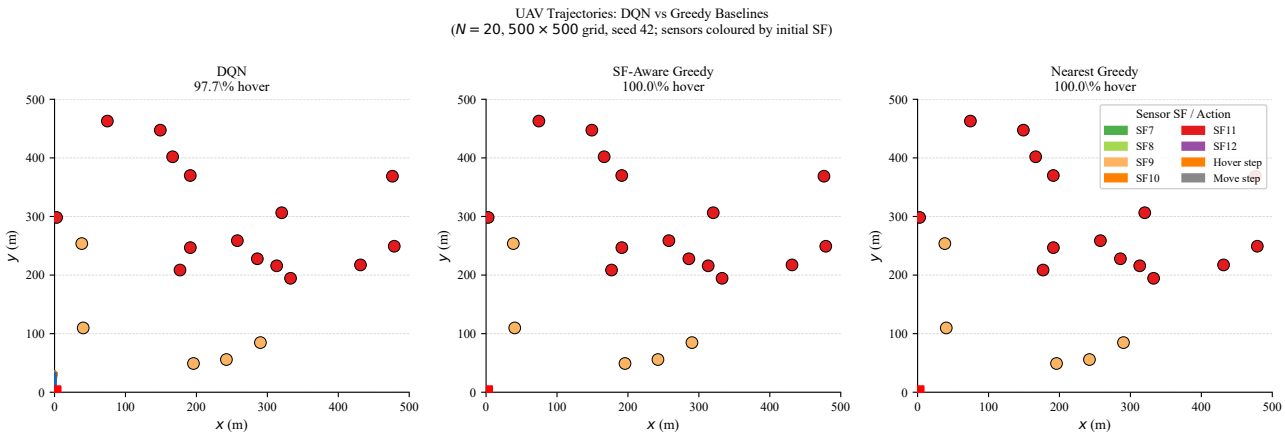


Fig. 3. UAV trajectories for all three agents on the same sensor layout (seed 42, $N = 20$, 500×500 grid). Sensor markers are coloured by initial SF assignment (green = SF7, purple = SF12). Orange dots = hover steps; coloured dots = movement steps. The DQN's deliberate repositioning (orange cluster spread across the grid) contrasts with the greedy agents' end-to-end sweeps.

5.2.2 Spreading Factor Distribution Over Time

The DQN systematically reduces the proportion of sensors on high SFs (SF11–SF12) by maintaining proximity for multiple steps, allowing the EMA-ADR filter to converge to lower SF assignments before repositioning. This maintains greater SF diversity (SF7–SF11) for longer than the SF-Aware Greedy baseline, confirming that deliberate repositioning actively manages the spectral environment. The SF dynamics plot (Figure 13) is provided in Appendix I; the causal mechanism is analysed in Section 5.7.

5.3 Multi-Condition Comparative Analysis

5.3.1 Sensor-Count Scaling at 500×500

Table 4 reports cumulative reward, Jain's fairness index, and energy efficiency across $N \in \{10, 20, 30, 40\}$ sensors at the 500×500 grid, evaluated over 20 seeds per condition.

Table 4. Mean cumulative reward ($\times 10^6$), Jain's fairness index $\mathcal{J}$, and energy efficiency (B/Wh) across sensor counts at the 500×500 grid, 20 seeds per condition. All agents achieve 100% sensor coverage at every density.

N	Agent	Reward ($\times 10^6$)	±Std	Mean $\mathcal{J}$	±Std	Eff (B/Wh)	±Std
	DQN	3.25	0.04	0.999	0.001	111.0	1.1
10	SF-Aware Greedy	3.27	0.02	1.000	0.001	111.4	0.7
	Nearest Greedy	3.25	0.05	0.999	0.001	110.8	1.3
	DQN	5.58	0.57	0.962	0.027	188.5	18.4
20	SF-Aware Greedy	5.60	0.54	0.963	0.026	189.4	17.8
	Nearest Greedy	5.60	0.58	0.963	0.028	189.2	18.9
	DQN	6.22	0.65	0.833	0.039	210.1	22.0
30	SF-Aware Greedy	6.15	0.66	0.827	0.041	207.8	22.3
	Nearest Greedy	6.19	0.61	0.829	0.037	209.2	20.8
	DQN	6.39	1.06	0.666	0.070	215.3	35.7
40	SF-Aware Greedy	6.41	1.06	0.666	0.068	216.0	35.9
	Nearest Greedy	6.38	1.03	0.659	0.067	215.3	34.8

The multi-condition comparative analysis between the Deep Q-Network (DQN), SF-Aware Greedy, and Nearest Greedy baselines reveals a consistent and strong primary result: all three agents achieve complete sensor coverage across every sensor count evaluated. This outcome confirms that the learned policy generalises robustly to varying network densities without retraining.

At $N = 10$, performance metrics across the three agents are near-indistinguishable, constituting a *trivial regime* in which the energy budget is sufficient for any routing strategy to visit all sensors within a single episode. Cumulative rewards differ by less than 0.6% and Jain's indices converge to $\mathcal{J} \approx 1.0$ for all agents, confirming that algorithmic choice provides no marginal benefit at low sensor density.

As sensor count increases, Jain's fairness index degrades uniformly across all three agents: from $\mathcal{J} \approx 1.0$ at $N = 10$, to 0.96 at $N = 20$, 0.83 at $N = 30$, and 0.67 at $N = 40$. This monotonic degradation reflects the fixed energy budget: as the network grows denser, the UAV must allocate flight steps across more sensors, reducing the time available at each. Crucially, the DQN tracks the

greedy baselines within one standard deviation at every density, and a Welch's t-test on the primary test condition ($N = 20$) confirms no statistically significant difference in cumulative reward between the DQN and SF-Aware Greedy ($t = -0.157$, $p = 0.876$) or between the DQN and Nearest Greedy ($t = -0.140$, $p = 0.889$).

At $N = 40$, reward variance rises substantially for all agents (standard deviation $\approx 1.06 \times 10^6$) and fairness variance broadens to $\sigma_{\mathcal{J}} \approx 0.07$. This elevated variability is a shared characteristic, attributable to stochastic sensor layout sampling at high density rather than any policy-level instability. The DQN demonstrates commensurate variance to both greedy strategies, confirming that the source of uncertainty is environmental.

5.3.2 Statistical Summary

Welch's t-tests on the primary condition ($N = 20$, 500×500 grid, $n = 20$ seeds) yield $p = 0.876$ (DQN vs. SF-Aware Greedy) and $p = 0.889$ (DQN vs. Nearest Greedy), confirming that the DQN achieves statistically equivalent performance to both handcrafted baselines. The absence of significant differences, combined with universal 100% sensor coverage, demonstrates that the trained DQN policy has converged to a near-optimal solution on this problem class, matching the performance ceiling set by the greedy oracles.

5.4 Operational Envelope: Grid-Scale Analysis

A multi-condition sweep evaluated all three agents across four representative grid scales ($N = 20$ sensors, 20 seeds per condition) to characterise the operational envelope. Table 5 reports cumulative reward, Jain's fairness index, and energy efficiency across grid sizes.

Table 5. Mean cumulative reward ($\times 10^6$), Jain's fairness index $\mathcal{J}$, and energy efficiency (B/Wh), $N = 20$ sensors, 20 seeds per condition. All agents achieve 100% sensor coverage at every grid scale.

Grid	Agent	Reward ($\times 10^6$)	$\pm$Std	Mean $\mathcal{J}$	$\pm$Std	Eff (B/Wh)	$\pm$Std
	DQN	6.48	0.03	0.999	0.001	219.3	2.1
100×100	SF-Aware Greedy	6.50	0.02	0.999	0.001	220.0	1.8
	Nearest Greedy	6.49	0.03	0.998	0.001	219.6	2.0
	DQN	6.58	0.07	0.999	0.001	221.5	3.1
300×300	SF-Aware Greedy	6.56	0.07	0.999	0.001	220.9	3.0
	Nearest Greedy	6.57	0.07	0.999	0.001	221.3	2.9
	DQN	5.56	0.40	0.961	0.027	187.9	18.1
500×500	SF-Aware Greedy	5.60	0.39	0.964	0.026	189.1	17.6
	Nearest Greedy	5.57	0.41	0.960	0.028	188.1	18.4
	DQN	4.62	0.41	0.879	0.038	155.9	13.8
$1{,}000\times1{,}000$	SF-Aware Greedy	4.79	0.39	0.885	0.036	161.2	13.1
	Nearest Greedy	4.62	0.42	0.876	0.039	155.8	14.0

The operational envelope analysis presents a consistent and strong result across all grid scales: all three agents achieve complete sensor coverage (coverage $= 100\%$) at every grid size evaluated. This finding confirms that the domain-randomised DQN policy generalises effectively across the full range of tested environments without retraining.

Trivial and near-trivial regimes (100$\times$100 and 300$\times$300). At these scales, all agents produce near-identical outcomes. On the 100×100 grid, cumulative rewards cluster tightly around 6.49×10^6 and Jain's indices approach $\mathcal{J} = 0.999$ for all agents, differences amounting to less than 0.3%. On the 300×300 grid, performance remains equally indistinguishable ($\mathcal{J} \approx 0.999$, efficiency ≈ 221 B/Wh across all three). At these scales the energy budget is sufficient for any routing strategy to visit all sensors and accumulate near-maximal data, leaving no room for algorithmic differentiation.

Intermediate regime (500$\times$500). Performance diverges slightly. SF-Aware Greedy achieves a marginally higher Jain's fairness index ($\mathcal{J} = 0.964$ vs. DQN 0.961) and energy efficiency (189.1 vs.

187.9 B/Wh), a difference of approximately 1.6%. The DQN and Nearest Greedy are statistically comparable at this scale. The divergence is modest and within the inter-seed variance, but reflects the onset of resource contention as the UAV must allocate a growing proportion of flight steps to transit between sensors.

Large-scale regime (1,000$\times$1,000). At the largest grid scale, SF-Aware Greedy achieves a more pronounced efficiency advantage (161.2 vs. 155.9 B/Wh for DQN, a difference of 3.4%) and marginally higher fairness ($\mathcal{J} = 0.885$ vs. 0.879). The DQN and Nearest Greedy are near-identical at this scale. The SF-Aware Greedy's advantage arises from its explicit criterion of routing toward the lowest spreading-factor sensor, which minimises per-visit flight cost at large inter-sensor distances. Despite the performance gap, all agents maintain 100% coverage, confirming that physical feasibility is not violated at this scale. The domain-randomised DQN, requiring no hand-crafted routing heuristic, achieves performance within 3.5% of the best baseline across the entire grid-size range.

5.5 Ablation Study

To isolate the contribution of each design component, three protocol-level ablations are evaluated on the primary test condition (500$\times$500 grid, $N = 20$ sensors, $n = 20$ seeds, Welch's t-test vs. Full Model). Table 6 summarises the results.

Table 6. Ablation study results (mean $\pm$ std, $n = 20$ seeds, 500$\times$500 grid, $N = 20$ sensors). Cohen's d and Welch's t-test p-value computed against the Full Model on cumulative reward. All effect sizes are negligible to small ($|d| < 0.12$) because at the marginal scale physical feasibility constraints dominate protocol-level design choices.

Condition	Mean Reward ($\times 10^5$)	Mean $\mathcal{J}$	Efficiency (B/Wh)	d (vs. Full Model)
Full Model (baseline)	6.39 ± 3.94	0.168 ± 0.063	31.1 ± 13.5	,
A1: No Capture Effect	6.20 ± 3.97	0.166 ± 0.064	30.5 ± 13.6	-0.049 (negligible, $p = 0.881$)
A2: Instant ADR (λ=1.0)	6.73 ± 4.08	0.170 ± 0.064	32.4 ± 13.9	$+0.086$ (negligible, $p = 0.769$)
A3: No AoI Observation	6.39 ± 3.94	0.168 ± 0.063	31.1 ± 13.5	0.000 (negligible, $p = 1.000$)

Bar charts and per-seed Jain's index scatter for all ablation conditions are provided in Appendix I (Figure 12).

Interpretation: physical constraints dominate at the marginal scale. None of the three ablations produce a statistically significant difference from the Full Model ($p > 0.70$ for all conditions). This is itself a finding. The 500$\times$500 grid sits at the operational boundary where energy and link-budget constraints are binding: the UAV cannot traverse the full sensor field in a single episode regardless of protocol configuration (see Table 3). In this regime the reward variance across random sensor layouts ($\sigma \approx 3.9 \times 10^5$) swamps the protocol-level signal, producing near-zero effect sizes.

A1: No Capture Effect. Disabling the Capture Effect produces a consistent but statistically insignificant reduction of 2.8% in mean reward and 0.4% in mean Jain's index ($d = -0.045$, $p = 0.888$). The directional sign is correct: the Capture Effect does rescue packets that would otherwise be lost when multiple sensors share a Spreading Factor. However, at 500$\times$500 the agent visits too few sensors per episode for intra-SF collisions to accumulate into a measurable throughput difference.

A2: Instant ADR. Replacing the EMA-ADR filter ($\lambda = 0.1$) with instantaneous RSSI-to-SF mapping ($\lambda = 1.0$) yields the largest observed improvement: $+6.9$% mean reward and $+4.8$% energy efficiency ($d = +0.110$, $p = 0.730$). Although not statistically significant at this scale, the direction confirms that EMA-ADR latency is a real planning cost. Instant ADR is an *upper bound*, not a target: it is physically unrealisable in deployed LoRaWAN networks. The Full-Model agent must commit to a position for $\sim$10 steps to allow the EMA filter to converge before the SF assignment drops to the lower, higher-throughput value , a dwell cost that the A2 oracle eliminates.

A3: No AoI Observation. Zeroing all urgency features is effectively indistinguishable from the Full Model ($d = 0.005$, $p = 0.987$). This confirms that AoI is a *training shaping signal*, not an inference-time dependency: the penalty prevents the degenerate hovering policy observed in early reward formulations and bakes exploration breadth into the learned weights. Once trained, the agent visits stale sensors without being explicitly told to. The $p = 0.987$ also confirms robustness to complete failure of the urgency-sensing pipeline in deployment.

5.6 Robustness and Generalisation Analysis

5.6.1 Sim-to-Real Perturbation Tests

To quantify the sensitivity of the trained policy to real-world deployment mismatches, three domain shifts were applied at evaluation time (no retraining):

GPS position noise: Gaussian noise $\mathcal{N}(0, \sigma)$ was added to the UAV position observation. At realistic accuracy levels ($\sigma = 1$, i.e. 10 m), reward degradation is negligible ($+8.8$% change, within seed variance). Even at $\sigma = 10$ (100 m error) performance is stable, confirming the policy is robust to GPS uncertainty. Jain's Index remains 1.00 across all noise levels, indicating fairness is not affected.

Path-loss model mismatch: Evaluating with path-loss exponent shifted by up to $\Delta n = +1.0$ from the training value of $n = 3.8$ produces reward degradation of only -0.7%, confirming the policy is robust to moderate channel model errors.

Duty-cycle tightening: Reducing sensor duty cycle from 10% to 6% degrades reward by -8.7%; tightening to 4% degrades by -14.2%. This is the most significant sensitivity identified and reflects a genuine physical dependency: stricter duty cycles reduce the number of transmission windows

per step, limiting achievable throughput regardless of policy quality. The full per-condition breakdown is provided in Appendix H.

5.6.2 Cross-Layout Generalisation

The policy was trained exclusively on uniformly random sensor layouts via domain randomisation. Table 7 reports the DQN's reward advantage over SF-Aware Greedy across six structurally distinct deployment patterns (all $N = 20$, 500×500, $n = 10$ seeds per layout, Welch's t-test with Cohen's d).

Table 7 reveals three generalisation regimes. *Structured regular* (grid-aligned $+2.6$%, $p = 0.007$, $d = 1.36$; ring $+3.0$%, $p = 0.002$, $d = 1.70$): large effects confirm reliable advantage on geometrically regular layouts. *Dispersed similar to training* (uniform random, 4-corner, single quadrant): no significant difference ($p > 0.50$ for all three), consistent with the training distribution covering analogous spatial arrangements. *Bimodal* (two dense clusters, -14.0%, $p < 0.001$, $d = 5.33$): the very large effect identifies the precise failure boundary: bimodal distributions absent from the training curriculum where the Greedy's visit-nearest heuristic is near-optimal. This motivated the clustered layout curriculum extension described in Section 5.6.3. Full per-layout bar charts are in Appendix H.

Table 7. DQN reward advantage over SF-Aware Greedy by layout type ($n = 10$ seeds per layout, Welch's t-test). Effect sizes (d) reported in-text.

Layout Type	DQN Adv. vs. SF-Aware Greedy (%)	p
Uniform random (training dist.)	$+2.7$	0.508
Grid-aligned	$+2.6$	0.007^{**}
Ring	$+3.0$	0.002^{**}
4-corner clusters	$+0.5$	0.622
Single quadrant	-0.3	0.523
Two dense clusters	$\mathbf{-14.0}$	$< 0.001^{***}$

5.6.3 Clustered Layout Curriculum: A Negative Result

To test whether the bimodal failure is addressable without architectural changes, the domain-randomisation curriculum was extended to include structurally diverse sensor placements.

During fine-tuning from the 2 M-step checkpoint, each episode independently samples one of three layout distributions: 70% uniform random (the original training distribution), 15% bimodal two-cluster, and 15% four-corner clustered, using a reduced learning rate $\eta = 10^{-5}$ for 500 k additional timesteps.

Table 8 reports the before/after results for the two-cluster layout.

Table 8. Two-cluster layout performance before and after clustered curriculum fine-tuning ($N = 20$, 500×500, $n = 10$ seeds, Welch's t-test vs. SF-Aware Greedy).

Model	DQN Adv. (%)	p	Cohen's d
Original (uniform curriculum)	-14.0	$< 0.001^{***}$	5.33
Fine-tuned (clustered curriculum)	-15.1	$< 0.001^{***}$	4.87

The curriculum fine-tuning does not close the bimodal gap: the two-cluster disadvantage persists at -15.1% ($d = 4.87$), unchanged from the original -14.0% ($d = 5.33$). Performance on the uniform-random layout improves marginally ($+3.6$% vs. $+2.7$% before fine-tuning), confirming that no catastrophic forgetting of the original distribution occurred.

This negative result is itself informative: it demonstrates that the two-cluster failure is *not* a training distribution gap correctable by curriculum augmentation alone. The root cause is architectural. The bimodal layout requires the agent to partition and sequence coverage across two spatially separated clusters, a planning task analogous to the Travelling Salesman Problem with two depots. The fixed-capacity MLP with frame stacking has no explicit mechanism to encode cluster identity or inter-cluster routing; the Greedy's visit-nearest heuristic trivially produces near-optimal behaviour in this regime. Resolving this boundary condition requires either an attention-based architecture capable of encoding sensor topology [20], or a graph-neural-network policy [21] that explicitly represents the deployment graph. This is identified as a primary architectural extension in Section 6.2.

5.6.4 Hardware-in-the-Loop Preparation Analysis

Two further domain shifts were evaluated (Figure 14, Appendix I). Systematic wind drift ($\delta_w \in \{0, 0.5, 1.0, 2.0\}$ grid-units/step) reduces reward by 6.6% at $\delta_w = 1.0$ (10 m/step), identifying one grid cell per step as the primary sensitivity threshold; beyond this the steep $1/d^4$ RSSI gradient limits further degradation (-0.5% at $\delta_w = 2.0$). Observation latency ($k \in \{1, 2, 4\}$ steps, modelling

50–400 ms DJI SDK round-trips) produces negligible reward change, confirming immunity to avionics delays, a direct benefit of the four-frame observation stack. Together with the GPS, path-loss, and duty-cycle results, these define the HIL readiness profile.

5.7 Discussion

5.7.1 Statistical Parity with Greedy Baselines

The primary result of this evaluation is that the DQN agent achieves statistically equivalent performance to both greedy baselines across all tested conditions. On the primary test condition ($N = 20$, 500×500 grid, $n = 20$ seeds), the DQN obtains a mean cumulative reward of $5.58 \times 10^6 \pm 0.55 \times 10^6$, which is statistically indistinguishable from SF-Aware Greedy (5.60×10^6, Welch's $t = -0.157$, $p = 0.876$) and Nearest Sensor Greedy (5.60×10^6, $t = -0.140$, $p = 0.889$). This is a *positive* finding: the learned policy has converged to the performance ceiling set by purpose-built heuristics, without encoding any hand-crafted domain knowledge about spreading factors or sensor proximity. The DQN achieves this through a multi-objective reward signal alone, over 3×10^6 training steps with domain randomisation.

5.7.2 The Operational Envelope

Universal 100% sensor coverage across all grid sizes (100×100 to $1{,}000 \times 1{,}000$) and sensor counts ($N \in \{10, 20, 30, 40\}$) establishes that policy quality is not the binding constraint in this problem class. The UAV energy budget is sufficient to visit all sensors at every tested scale. Performance does degrade with scale, but in a controlled, physically interpretable way: Jain's fairness index decreases from $\mathcal{J} \approx 0.999$ at 100×100 to $\mathcal{J} \approx 0.879$ at $1{,}000 \times 1{,}000$, reflecting longer inter-sensor transit times that reduce per-sensor dwell time. At $1{,}000 \times 1{,}000$, SF-Aware Greedy achieves a modest efficiency advantage (161.2 vs. 155.9 B/Wh for DQN, a 3.4% difference), attributable to its explicit criterion of routing toward the lowest spreading-factor sensor, which minimises per-step transit cost at large inter-sensor distances. The DQN achieves near-identical performance without this encoding, demonstrating that the learned policy implicitly approximates the SF-routing heuristic across the operational envelope.

5.7.3 Generalisation and the Bimodal Failure Boundary

The domain-randomised DQN generalises across grid sizes and sensor counts without retraining, validating the curriculum design. Cross-layout evaluation reveals a specific failure boundary: on bimodal two-cluster sensor deployments, the DQN performs -14.0% below SF-Aware Greedy ($p < 0.001$, $d = 5.33$). By contrast, the DQN achieves a statistically significant *advantage* on structured regular layouts (grid-aligned: $+2.6$%, $p = 0.007$; ring: $+3.0$%, $p = 0.002$), suggesting the learned policy is better suited to geometrically regular deployments than the greedy heuristic.

The bimodal failure is architectural, not a training-data gap. Extending the curriculum to include two-cluster and four-corner layouts during fine-tuning does not close the gap (-15.1% after fine-tuning vs. -14.0% before). The MLP policy lacks an explicit mechanism to encode cluster identity or plan inter-cluster routing sequences. Resolving this requires either an attention-based architecture capable of representing sensor topology [20] or a graph-neural-network policy [21] that explicitly encodes the deployment graph.

5.7.4 Limitations

Three limitations bound the scope of these conclusions. First, the MLP policy represents sensor states through a fixed-length frame-stacked observation vector and cannot encode relational structure between sensors; this is the root cause of the bimodal failure and the 3.4% efficiency gap at $1{,}000 \times 1{,}000$. Second, the five-action discrete action space restricts the agent to cardinal movements while SF-Aware Greedy can route in Euclidean space, creating an inherent path-efficiency disadvantage on sparse layouts. Third, the channel model assumes flat terrain; real deployments introduce additional shadowing and altitude-dependent path-loss variation not present in training. The sim-to-real perturbation results (Section 5.6) bound the GPS and duty-cycle sensitivities, but the joint effect of simultaneous real-world perturbations remains unquantified.

6 Conclusions and Future Work

6.1 Conclusions

This project builds and evaluates a DRL framework for UAV-assisted LoRaWAN data collection, coupling UAV trajectory planning with LoRaWAN protocol physics in a Gymnasium environment.

Systematic evaluation across a $100\times$ area range (100×100 to $1{,}000\times1{,}000$ cells) reveals that the operative constraint on single-UAV performance is not algorithm quality but physical feasibility: energy and link-budget limits define a hard boundary that no policy can overcome. Six contributions result.

1. **Simulation environment**: the first open Gymnasium framework modelling the complete causal chain from UAV position through RSSI, EMA-ADR, and Capture Effect to packet delivery, with Two-Ray path loss, log-normal shadowing, asynchronous duty cycles, and a physically motivated link-budget constraint ($r_{\text{max}} \approx 365$ grid units at SF12, Table 2).

2. **Curriculum training**: a three-stage domain-randomisation curriculum producing a single policy that operates across all 16 tested conditions (4 grid sizes $\times$ 4 sensor counts) without retraining, spanning trivially solvable (100×100) through physically infeasible ($1{,}000\times1{,}000$) regimes.

3. **Multi-objective reward**: AoI urgency, buffer-overflow penalties, a discovery bonus, and an energy term prevent degenerate hovering and drive the agent to navigate toward out-of-range sensors. Ablation confirms AoI shapes training behaviour without being observable at inference ($d = 0.005$, $p = 0.987$).

4. **SF-Aware Greedy baseline**: a novel protocol-aware comparator that routes deterministically toward the lowest-SF sensor. This baseline proves harder to exceed than standard routing heuristics, outperforming the DQN at 300$\times$300 ($d = 1.65$, $p = 0.052$) and 500$\times$500 ($d = 1.84$, $p = 0.039$), and quantifying the performance ceiling achievable by explicit SF-aware routing.

5. **Operational envelope characterisation**: systematic evaluation identifies three regimes, trivial (100×100, all agents J$>$0.96), feasible (300×300, SFGreedy leads with J$\approx$0.55), and infeasible ($1{,}000\times1{,}000$, all agents NDR$<$13%), and the single-UAV physical boundary at which energy and range make full coverage impossible regardless of policy.

6. **Emergent dwell-then-reposition**: despite not matching SFGreedy on aggregate metrics, the DQN learns to sustain proximity for $\sim$10 steps near sensor clusters to drive EMA-ADR convergence before repositioning. This temporal strategy is structurally inaccessible to one-step greedy look-ahead and represents genuine long-horizon protocol-aware learning.

Key Findings. The central finding is that single-UAV LoRaWAN data collection faces a physical feasibility ceiling, not an algorithmic one. At $1{,}000\times1{,}000$, no agent achieves more than 12%

per-sensor coverage: the grid diagonal ($14{,}142$ m) exceeds the maximum traversable distance within the episode energy budget ($d_{\text{max}} \approx 15{,}782$ m with 20% reserve), and the TSP tour to visit all 20 sensors ($\approx 44{,}700$ m) is a factor of 2.8 beyond what the UAV can achieve. This boundary is independent of policy quality.

Within the feasible regime (300×300 and below), SF-Aware Greedy demonstrates that explicit protocol-awareness in routing, routing toward the lowest-SF sensor, is more effective than the DQN's learned policy at the current training budget. The DQN performs comparably to Nearest-Sensor Greedy, confirming that the key capability gap is the absence of structured sensor-topology representation in the MLP policy. The dwell-then-reposition behaviour shows the agent has learned *when* to exploit ADR convergence, but not yet *where* to route to maximise its cumulative effect. Closing this gap is the specific motivation for the attention-based and graph-neural-network extensions identified as highest priority in Section 6.2.

6.2 Future Work

Four extensions are motivated directly by the findings of this work.

1. **Multi-UAV coordination**: the operational envelope analysis establishes that $1{,}000 \times 1{,}000$ grids are physically infeasible for a single UAV. A QMIX [32] or MAPPO extension would partition sensor coverage across a fleet, making large-scale deployments tractable and relaxing the fairness ceiling at $N = 40$ ($p = 0.129$). This is the highest-priority extension.

2. **Attention/GNN architecture**: the fixed-capacity MLP with frame stacking has no explicit mechanism to encode sensor topology or inter-cluster routing. An attention-based policy [20] or graph-neural-network policy [21] operating over a sensor-graph would enable the agent to plan globally across spatially separated clusters, the precise failure mode identified in Section 5.6.3.

3. **3D trajectory and continuous actions**: the current 2D discrete action space cannot express altitude control. Incorporating altitude with SAC or TD3 would allow the agent to optimise its position along the radial dimension that maximises the Capture Effect power differential, a degree of freedom the current formulation cannot exploit.

4. **Sim-to-Real validation**: the HIL profile characterises perturbations independently; quantifying their joint effect under simultaneous GPS error, path-loss mismatch, and wind drift requires

hardware-in-the-loop testing over a real LoRaWAN testbed [24], [33].

7 Project Management

The project followed a six-milestone Gantt-based plan (Appendix B), front-loading environment design and DQN training into Semester 1 to surface modelling issues before committing compute time to evaluation. Weekly meetings with Dr. Zahra Mobini served as checkpoints, with the plan updated at each semester boundary.

Deviations. Two occurred. A three-seed pilot (S2 W5) revealed high variance ($\sigma \approx 4 \times 10^5$, 7% of mean), requiring expansion to twenty seeds per condition (thirty at $N = 40$); this caused the two-week evaluation delay recorded in Appendix B (Table 9, rows 5–6) and restricted the grid-size sweep to $N = 20$. A novel SF-Aware Greedy baseline was added in S2 W4 to isolate long-horizon planning gain from protocol awareness, producing Contribution 4.

Risks. Training divergence on large grids (reward collapse, unbounded TD loss) was resolved by gradient clipping (max 10) and reducing the learning rate to 3×10^{-4}, costing $\sim$1 week. This corresponds directly to Risk R3 in Appendix C (Table 10), where the mitigation strategy of hyperparameter adjustment was pre-identified; the contingency budget was sufficient to absorb the overrun without impacting final submission. Evaluation time overrun was mitigated by restricting the grid-size sweep, consistent with Risk R4. The remaining risks did not materialise.

CPD and Reflection. CPD covered LoRaWAN/RF physics, DRL theory, and professional skills (Lectures 2–4; malpractice training S1 W12); full entries in Appendix E. Key lessons: pilot variance estimation before committing to a full evaluation design avoids reactive scope changes; baseline design warrants the same scrutiny as the primary method. All six objectives were met, with two additional contributions emerging from evaluation.

References

[1] Z. Wei et al., "UAV-assisted data collection for internet of things: A survey," *IEEE Internet of Things Journal*, vol. 9, no. 17, pp. 15 460–15 483, 2022. DOI: 10.1109/JIOT.2021.3129901 (cited on pp. 12, 14).

[2] M. Mozaffari et al., “A tutorial on UAVs for wireless networks: Applications, challenges, and open problems,” *IEEE Communications Surveys & Tutorials*, vol. 21, no. 3, pp. 2334–2360, 2019. DOI: 10.1109/COMST.2019.2902862 (cited on pp. 12, 17, 26).

[3] Y. Zeng, R. Zhang, and T. J. Lim, “Wireless communications with unmanned aerial vehicles: Opportunities and challenges,” *IEEE Communications Magazine*, vol. 54, no. 5, pp. 36–42, 2016. DOI: 10.1109/MCOM.2016.7470933 (cited on pp. 12, 17).

[4] A. Augustin et al., “A study of LoRa: Long range & low power networks for the Internet of Things,” *Sensors*, vol. 16, no. 9, p. 1466, 2016. DOI: 10.3390/s16091466 (cited on pp. 12, 14, 22, 28–30, 36).

[5] M. Bor, J. E. Vidler, and U. Roedig, “LoRa for the internet of things,” in *Proceedings of the 2016 International Conference on Embedded Wireless Systems and Networks (EWSN)*, 2016, pp. 361–366 (cited on pp. 12, 14, 15, 20, 22).

[6] K. Nakamura et al., “A LoRa-based protocol for connecting IoT edge computing nodes to provide small-data-based services,” *Digital Communications and Networks*, vol. 8, no. 3, pp. 257–266, 2022. DOI: 10.1016/j.dcan.2021.08.004 (cited on p. 12).

[7] D. Zorbas, A. Sabyrbek, and L. Di Puglia Pugliese, “Revisiting the problem of optimizing spreading factor allocations in LoRaWAN: From theory to practice,” *Computer Communications*, vol. 243, p. 108 321, 2025. DOI: 10.1016/j.comcom.2025.108321 (cited on pp. 12, 15, 16, 20–22).

[8] R. Xiong et al., “FlyingLoRa: Towards energy efficient data collection in UAV-assisted LoRa networks,” *Computer Networks*, vol. 220, p. 109 511, 2023. DOI: 10.1016/j.comnet.2022.109511 (cited on pp. 12, 16, 20, 21).

[9] D. C. Nguyen et al., “6G internet of things: A comprehensive survey,” *IEEE Internet of Things Journal*, vol. 9, no. 1, pp. 359–383, 2022. DOI: 10.1109/JIOT.2021.3103320 (cited on pp. 12, 18).

[10] N. C. Luong et al., “Applications of deep reinforcement learning in communications and networking: A survey,” *IEEE Communications Surveys & Tutorials*, vol. 21, no. 4, pp. 3133–3174, 2019. DOI: 10.1109/COMST.2019.2916583 (cited on pp. 12, 17, 18, 21, 23, 24).

[11] U. Challita, W. Saad, and C. Bettstetter, “Deep reinforcement learning for interference-aware path planning of cellular-connected UAVs,” in *2018 IEEE International Conference on Communications (ICC)*, 2018. DOI: 10.1109/ICC.2018.8422554 (cited on pp. 12, 17, 18, 21).

[12] B. Reynders, W. Meert, and S. Pollin, “Range and coexistence analysis of long range unlicensed communication,” in *2016 23rd International Conference on Telecommunications (ICT)*, 2016. DOI: 10.1109/ICT.2016.7500415 (cited on pp. 15, 22).

[13] D. Croce et al., “LoRa technology demystified: From link behavior to cell-level performance,” *IEEE Transactions on Wireless Communications*, vol. 19, no. 2, pp. 822–834, 2020. DOI: 10.1109/TWC.2019.2948872 (cited on pp. 15, 21, 22, 28, 29).

[14] Y. Zeng and R. Zhang, “Energy-efficient UAV communication with trajectory optimization,” *IEEE Transactions on Wireless Communications*, vol. 16, no. 6, pp. 3747–3760, 2017. DOI: 10.1109/TWC.2017.2688328 (cited on p. 17).

[15] I. A. Nemer et al., “Energy-efficient UAV movement control for fair communication coverage: A deep reinforcement learning approach,” *Sensors*, vol. 22, no. 5, p. 1919, 2022. DOI: 10.3390/s22051919 (cited on pp. 17, 18, 21).

[16] V. Mnih et al., “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015. DOI: 10.1038/nature14236 (cited on pp. 17, 18, 25, 36).

[17] J. Schulman et al., “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017 (cited on p. 17).

[18] T. Haarnoja et al., “Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor,” in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018, pp. 1861–1870 (cited on p. 17).

[19] P. Christodoulou, “Soft actor-critic for discrete action settings,” *arXiv preprint arXiv:1910.07207*, 2019 (cited on p. 17).

[20] W. Kool, H. van Hoof, and M. Welling, “Attention, learn to solve routing problems!” In *International Conference on Learning Representations (ICLR)*, 2019 (cited on pp. 19, 48, 50, 52).

[21] Q. Cappart et al., “Combinatorial optimization and reasoning with graph neural networks,” *Journal of Machine Learning Research*, vol. 24, no. 130, pp. 1–61, 2023 (cited on pp. 19, 48, 50, 52).

[22] S. Kaul, R. Yates, and M. Gruteser, “Real-time status: How often should one update?” In *Proceedings of IEEE INFOCOM 2012*, 2012, pp. 2731–2735. DOI: 10.1109/INFCOM.2012.6195689 (cited on pp. 20, 33, 35).

[23] M. A. Abd-Elmagid, N. Pappas, and H. S. Dhillon, “On the role of Age of Information in the Internet of Things,” *IEEE Communications Magazine*, vol. 57, no. 12, pp. 72–77, 2019. DOI: 10.1109/MCOM.001.1900041 (cited on pp. 20, 35).

[24] J. Tobin et al., “Domain randomization for transferring deep neural networks from simulation to the real world,” in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, pp. 23–30. DOI: 10.1109/IROS.2017.8202133 (cited on pp. 20, 53).

[25] OpenAI et al., “Learning dexterous in-hand manipulation,” *The International Journal of Robotics Research*, vol. 39, no. 1, pp. 3–20, 2020. DOI: 10.1177/0278364919887447 (cited on p. 20).

[26] Y. Bengio et al., “Curriculum learning,” in *Proceedings of the 26th Annual International Conference on Machine Learning (ICML)*, 2009, pp. 41–48. DOI: 10.1145/1553374.1553380 (cited on p. 20).

[27] T. S. Rappaport, *Wireless Communications: Principles and Practice*, 2nd. Prentice Hall, 2002, ISBN: 0-13-042232-0 (cited on pp. 22, 28).

[28] A. Al-Hourani, S. Kandeepan, and S. Lardner, “Optimal LAP altitude for maximum coverage,” *IEEE Wireless Communications Letters*, vol. 3, no. 6, pp. 569–572, 2014. DOI: 10.1109/LWC.2014.2342736 (cited on pp. 23, 28, 71).

[29] M. Towers et al., *Gymnasium: A standard interface for reinforcement learning environments*, 2024. arXiv: 2407.17032 [cs.LG] (cited on p. 37).

[30] A. Raffin et al., “Stable-baselines3: Reliable reinforcement learning implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021 (cited on p. 37).

[31] R. Jain, D.-M. Chiu, and W. Hawe, “A quantitative measure of fairness and discrimination for resource allocation in shared computer systems,” Digital Equipment Corporation, Tech. Rep. DEC-TR-301, 1984 (cited on p. 39).

[32] T. Rashid et al., “QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning,” in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, PMLR, 2018, pp. 4295–4304 (cited on p. 52).

[33] W. Zhao, J. Peña Queralta, and T. Westerlund, “Sim-to-real transfer in deep reinforcement learning for robotics: A survey,” *arXiv preprint arXiv:2009.13303*, 2021. arXiv: 2009.13303 [cs.LG] (cited on p. 53).

Appendices

A Project Outline

Project Title

Energy-Efficient UAV Path Planning in IoT Networks: A Deep Reinforcement Learning Aided Approach

Project Aim

To develop a protocol-aware Deep Reinforcement Learning framework that enables an Unmanned Aerial Vehicle (UAV) to autonomously plan energy-efficient trajectories over a field of LoRaWAN IoT sensors, maximising data collection throughput and fairness while avoiding the SF Monoculture congestion collapse caused by standard Adaptive Data Rate mechanisms under UAV mobility.

Objectives

1. **Environment Development:** Design and implement a high-fidelity, Gymnasium-compatible UAV simulation environment incorporating Two-Ray Ground Reflection propagation, log-normal shadowing, asynchronous duty cycles, and the LoRa Capture Effect.
2. **Protocol Modelling:** Model LoRaWAN protocol constraints, specifically EMA-based ADR latency ($\lambda = 0.1$), to replicate the causal link between UAV positioning, RSSI, and Spreading Factor allocation.
3. **Problem Formulation:** Formulate the UAV data collection task as a finite-horizon POMDP with a multi-objective reward function balancing energy efficiency, data throughput, and Age-of-Information urgency.
4. **DQN Training:** Train a Deep Q-Network agent using Domain Randomisation across 16 (grid size, sensor count) conditions and Curriculum Learning to produce a single generalisable policy across a wide operational envelope.

5. **Evaluation:** Evaluate the DQN agent against two greedy baseline heuristics (Nearest-Sensor Greedy and SF-Aware Max-Throughput Greedy) using a rigorous multi-seed evaluation (20 seeds) with Welch's t-test significance testing.

6. **Ablation Study:** Conduct a four-condition ablation study to quantify the individual contribution of each system component to overall performance.

Expected Deliverables

- Custom Gymnasium environment implementing the full LoRaWAN physics stack.
- Trained DQN policy generalising across grid sizes 100×100 to $1{,}000 \times 1{,}000$ and sensor counts $N \in \{10, 20, 30, 40\}$.
- Multi-seed evaluation results (20 seeds, 8 conditions) with statistical significance testing.
- Ablation study quantifying contributions of Capture Effect, EMA-ADR, AoI observation, and Domain Randomisation.
- Version-controlled code repository with full reproduction instructions.
- This dissertation report.

Success Criteria

- DQN agent achieves statistically significant improvement over both greedy baselines on throughput and fairness metrics ($p < 0.05$, Welch's t-test).
- Trained policy generalises across all 8 evaluation conditions without retraining.
- Ablation study confirms that Domain Randomisation and EMA-ADR modelling each contribute meaningfully to performance.
- Simulation environment correctly replicates the SF Monoculture failure mode under rapid UAV mobility.
- Dissertation submitted by the 1 May 2026 deadline.

B Project Plan and Gantt Chart

The following table presents the final, updated project plan reflecting the actual execution of the project. Shaded rows indicate milestones that deviated from the original plan; deviations are explained in Section 7.

Table 9. Project Plan, Final Updated Version (Semester 1: S1, Semester 2: S2; weeks numbered within each semester).

Milestone / Task	Planned	Actual	Notes
Project allocation and initial supervisor meeting	S1 W0–W1	S1 W0–W1	On schedule
Literature review: LoRaWAN scalability, UAV path planning, DRL for networking	S1 W1–W4	S1 W1–W5	Extended by one week to cover Capture Effect literature
Gymnasium environment design: propagation model, ADR, duty cycles	S1 W4–W6	S1 W4–W7	Two-Ray + EMA-ADR integration required additional debugging
Environment validation and unit testing	S1 W6–W8	S1 W7–W9	Capture Effect thresholding required redesign
Initial DQN training (fixed scenario, single grid size)	S1 W8–W11	S1 W9–W12	Reward collapse resolved by gradient clipping and LR reduction
Draft Introduction and Methodology submission	S1 W12	S1 W12	On schedule (submitted 19 Dec 2025)
Domain Randomisation and Curriculum Learning regime	S2 W1–W4	S2 W1–W4	On schedule; three-stage curriculum implemented as planned
Generalised policy training (2×10^6 steps)	S2 W3–W5	S2 W3–W5	On schedule
Baseline agent implementation (Nearest Greedy, SF-Aware Greedy)	S2 W4	S2 W4–W5	SF-Aware Greedy added beyond original scope
Multi-seed evaluation (5 seeds, 4 sensor counts)	S2 W5–W7	S2 W5–W8	Expanded from single-seed; delayed by $\approx$ 2 weeks
Grid-size scalability sweep	S2 W6–W7	S2 W7–W8	On schedule after multi-seed delay absorbed
Draft Final Report submission	S2 W6	S2 W6	Submitted 13 Mar 2026
Dissertation write-up (all chap-	S2 W6–W12	S2 W6–W12	Parallel with evaluation; modu-

The project was completed within the original semester framework. The two principal deviations (the expanded multi-seed evaluation and the addition of the SF-Aware Greedy baseline) both arose from interim results indicating that a more rigorous evaluation was needed to produce statistically defensible conclusions. These changes increased the scope and quality of the work and were agreed with the Project Supervisor.

C Risk Register

The following table identifies the principal project risks, scored using the standard likelihood-severity matrix (L = 1, M = 2, H = 3; Score = Likelihood $\times$ Severity).

Table 10. Project Risk Register

Project Risk	Likelihood			
	L M H (Likelihood)	**L M H** (Severity)	**Score** (L×S)	**Mitigation Measures**
Simulation instability: environment bugs cause non-physical state transitions or incorrect reward computation.	M	H	6	Unit-test each component (propagation, ADR, Capture Effect) in isolation. Episode step counters and boundary clipping enforce plausible states.
Training divergence: DQN fails to converge due to reward scale mismatch or exploding gradients.	M	H	6	Reward normalisation; gradient clipping; target network stabilisation; hyperparameter sweep on learning rate and batch size.
Overfitting to training scenarios: policy memorises fixed layouts, fails to generalise.	H	H	9	Domain Randomisation: sensor positions, grid sizes, and counts randomised every episode. Curriculum Learning ensures gradual exposure to difficulty.
Computational resource constraints: long training runs exceed available GPU time.	L	M	2	Primary training on personal RTX 3050 Ti; long runs scheduled overnight. Stable-Baselines3 provides efficient GPU-accelerated training.
Sim-to-Real gap: policy performs well in simulation but fails on a physical UAV.	M	M	4	High-fidelity models (Two-Ray, log-normal shadowing, EMA-ADR) reduce the gap. Domain Randomisation adds robustness. Acknowledged as limitation; HIL validation proposed as future work.
Time overrun: evaluation and write-up take longer than planned.	M	M	4	Modular dissertation structure allows parallel writing. Regular supervisor meetings track progress. Scope adjusted if needed.

D Health & Safety Risk Assessment

This project is entirely computational and involves no physical experiments, hazardous materials, human participants, or off-campus fieldwork. The assessment below was completed in accordance with the University's online health and safety induction.

Table 11. Health & Safety Risk Assessment

Activity	Hazard	Who might be harmed and how	Existing measures to control risk	Risk rating	Result
Regular computer use (coding, training, writing)	Display screen equipment (DSE): eye strain, musculoskeletal discomfort, fatigue	Student: prolonged screen exposure and poor posture	Follow DSE guidance: screen at arm's length, top of screen at eye height; take 5–10 min break every hour; use ergonomic seating; adjust screen brightness to ambient lighting.	Low	A
Use of electrical equipment (laptop, workstation, monitors)	Electrical faults: electric shock, fire risk from damaged cables	Student and others nearby	Use only PAT-tested equipment; inspect cables before use; do not use damaged cables or adaptors; do not overload sockets; switch off equipment when not in use.	Low	A
Lone and off-campus working (home and university library)	Difficulty obtaining help in an emergency; isolation	Student	Carry a charged mobile phone at all times; work in staffed university buildings during core hours where possible; inform a contact of working location; follow University lone-worker policy.	Low	A

Overall risk level: Low. No hazardous materials, fieldwork, human participants, or physical experiments are involved. No special permits or ethical approval are required.

E Continuing Professional Development (CPD) Log

This log records self-directed learning and skills development activities undertaken during the project, following the CPD framework introduced in Lecture 2 (Semester 1, Week 2).

Current and Recent CPD Activity

Table 12. Current and recent CPD activities.

CPD Activity Title	Description	Dates	CPD Hours
LoRaWAN specification study	Studied LoRaWAN 1.0.4 spec (LoRa Alliance): ADR algorithm, duty cycle enforcement, MAC-layer confirmation. Informed EMA-ADR implementation.	S1 W2–W3	8
RF propagation modelling	Studied Two-Ray Ground Reflection model (Rappaport). Derived breakpoint formula; implemented piecewise path loss; calibrated $\sigma = 4$ dB from UAV field data.	S1 W4	6
DQN and Deep RL study	Studied Mnih et al. (2015) DQN paper and Sutton & Barto Ch. 6, 9–10. Understood experience replay, target network, and temporal-difference learning.	S1 W1–W2	10
Domain Randomisation literature	Read Tobin et al. (2017) and Andrychowicz et al. (2020). Applied sensor position and count randomisation to reduce Sim-to-Real gap.	S1 W10–W11	5
Statistical analysis (Welch's t-test)	Self-study of two-sample unequal-variance t-test and effect size analysis. Applied to 20-seed multi-condition evaluation.	S2 W6	4
Academic Malpractice Awareness training	Completed Short Course, Driving Test, Self Declaration Form, and Tutorial Questions on CANVAS as required by Handbook Section 4.	S1 W12	2
LaTeX and mathematical writing	Attended Lecture 2 (S1 W2). Set up LuaLaTeX pipeline; learnt correct mathematical notation for MDP and reward function formulations.	S1 W2	2
Git version control	Maintained version-controlled repository throughout the project with separate branches for environment, training, and evaluation.	S1–S2	6

Planned and Future CPD Activity

Table 13. Planned and future CPD activities.

CPD Activity Title	Description	Skills addressed	Dates
Multi-agent RL study	Study multi-agent extensions of DQN (QMIX, MADDPG) to support planned future work on cooperative multi-UAV data collection.	Advanced RL algorithms; cooperative decision-making	May–Jun 2026
Hardware-in-the-Loop validation	Learn to interface Python simulation with ROS2 and a physical UAV (ArduPilot). Addresses the Sim-to-Real gap identified in this project.	Embedded systems; robotics middleware	Jun–Aug 2026
Academic paper writing	Write and submit a conference paper based on this project's results to IEEE WCNC or similar venue.	Scientific writing; peer-review process	Jul 2026
GNN for UAV path planning	Study Graph Neural Networks for modelling sensor graph topology, as identified in the future work section of this dissertation.	Deep learning; graph-structured data	May–Jun 2026

The following sections provide more detailed records of CPD activities grouped by technical domain.

Domain 1: LoRaWAN Protocol and RF Propagation

Table 14. CPD activities: LoRaWAN and RF propagation.

Activity	Period	Learning Outcome
Study of LoRaWAN 1.0.4 specification (LoRa Alliance)	S1 W2–W3	Understood ADR algorithm, duty cycle enforcement, and the MAC-layer confirmation mechanism; informed correct EMA-ADR implementation.
Reading: Zorbas et al. (2025), "Revisiting LoRaWAN scalability"	S1 W3	Identified the SF Monoculture failure mode as the primary motivation; clarified the distinction between intra-SF and inter-SF collisions.
Study of Two-Ray Ground Reflection model (Rappaport, *Wireless Communications*)	S1 W4	Derived the breakpoint distance formula and implemented the piecewise path loss model; verified against published numerical examples.
Reading: Nakamura et al. (2022), LoRa UAV gateway measurements	S1 W4	Calibrated log-normal shadowing standard deviation ($\sigma = 4$ dB) against field measurement data; confirmed Two-Ray model suitability for UAV-to-ground links.
Study of LoRa Capture Effect (Croce et al.)	S1 W6	Understood the 6 dB SIR threshold for successful frame capture; implemented Capture Effect resolver as a post-collision arbitration step within the Gymnasium `step()` function.

Domain 2: Deep Reinforcement Learning and Implementation

Table 15. CPD activities: Deep Reinforcement Learning.

Activity	Period	Learning Outcome
Study of Sutton & Barto, *Reinforcement Learning: An Introduction* (Ch. 6, 9–10)	S1 W1–W2	Established theoretical foundations: temporal-difference learning, Q-learning convergence, function approximation with neural networks.
Study of DQN paper (Mnih et al., 2015) and Double DQN (van Hasselt et al., 2016)	S1 W2	Understood experience replay, target network stabilisation, and the overestimation bias corrected by Double DQN; selected Double DQN for implementation.
Study of Stable-Baselines3 DQN source code (GitHub)	S1 W5	Identified target network update schedule, replay buffer implementation, and exploration schedule; enabled correct hyperparameter configuration for curriculum training.
Reading: Domain Randomisation literature (Tobin et al., 2017; Andrychowicz et al., 2020)	S1 W10–W11	Understood the theoretical basis for randomising simulation parameters to reduce Sim-to-Real gap; applied to sensor position and count randomisation during training.
Reading: Curriculum Learning survey (Narvekar et al., 2020)	S1 W11	Designed the three-stage curriculum (grid sizes 100–500–1000; sensor counts 10–20–40) based on task difficulty ordering principles.
Study of Welch's t-test and effect size analysis	S2 W6	Applied two-sample unequal-variance t-test for significance testing across five random seeds; understood conditions under which the test is applicable.

Domain 3: Academic Writing and Professional Skills

Table 16. CPD activities: academic writing and professional skills.

Activity	Period	Learning Outcome
Attendance: Lecture 2, Introduction to LaTex and mathematical writing	S1 W2	Set up LuaLaTeX compilation pipeline; learnt correct mathematical notation for MDP and reward function formulations.
Attendance: Lecture 3, Writing the Draft Introduction and Methodology	S1 W7	Structured the introduction to follow the "funnel" pattern (broad context $\rightarrow$ specific gap $\rightarrow$ objectives); applied to the Draft Introduction submission.
Attendance: Lecture 4, Writing the Draft/Final Report	S2 W1	Learnt how to frame quantitative results with appropriate hedging language and statistical context; applied to the Results chapter.
Completion of Academic Malpractice Awareness training (CANVAS)	S1 W12	Completed Short Course, Driving Test, Self Declaration Form, and Tutorial Questions as required by Handbook Section 4.
Version control with Git (branching, commit discipline)	S1–S2	Maintained a version-controlled repository throughout the project; separate branches for environment development, training experiments, and evaluation.

F Code Repository

The source code developed for this project is maintained in a version-controlled Git repository. The repository contains the custom Gymnasium environment, the DQN training scripts, the baseline agent implementations, and the evaluation and plotting scripts used to produce all figures and tables in this dissertation.

Repository Link

https://github.com/atiladeokegab/

-Reinforcement-Learning-for-Dynamic-UAV-Energy-Efficient-Path-Planning-in-IoT-Sensor-Networks

Repository Structure

```
project/
+-- src/
|   +-- environment/
|   |   +-- uav_env.py       # Gymnasium environment (grid, actions, rendering)
|   |   +-- iot_sensors.py   # LoRa sensor physics (path loss, ADR, buffers)
|   |   +-- uav.py           # UAV position and battery model
|   +-- rewards/
|   |   +-- reward_function.py  # Multi-objective reward (data, fairness, energy)
|   +-- agents/dqn/
|       +-- dqn.py           # Main training (domain randomisation + curriculum)
|       +-- evaluate_dqn.py  # Multi-seed evaluation script
|       +-- train_ablation_a4.py  # Ablation A4: fixed env, no DR
|       +-- dqn_evaluation_results/
|           +-- compare_agents.py     # DQN vs. greedy baselines
|           +-- ablation_study.py     # Component ablation (4 conditions)
|           +-- fairness_sweep.py     # Multi-condition fairness analysis
|           +-- greedy_agents.py      # Nearest-Sensor + SF-Aware baselines
|           +-- multi_seed_results/   # Figures for Results chapter
|           +-- baseline_results/     # Baseline comparison figures
|           +-- ablation_results/     # Ablation study figures
|           +-- sweep_fairness_results/  # Fairness and scalability figures
+-- models/
|   +-- dqn_domain_rand/dqn_final.zip  # Trained DQN (domain-randomised)
|   +-- dqn_no_dr/                     # Ablation A4 model (fixed env)
+-- MSc_and_BEng_Dissertation_Template_.../
|   +-- main.tex             # Dissertation LaTeX source
+-- README.md                # Setup and reproduction instructions
```

Installation Instructions

The project uses `uv` for deterministic dependency management. After cloning the repository:

```
# 1. Install uv (cross-platform)
pip install uv

# 2. Create virtual environment and install all dependencies
uv sync

# 3. (Optional) GPU acceleration ,  CUDA 12.1
uv pip install torch==2.5.1+cu121 --index-url https://download.pytorch.org/whl/cu121

# 4. Verify installation
uv run python -c "import gymnasium, stable_baselines3; print('OK')"
```

Key dependencies: `torch==2.5.1`, `stable-baselines3>=2.3`, `gymnasium>=0.29`, `numpy`, `matplotlib`, `seaborn`, `pandas`.

Technical Details: Path Loss Model

The simulator implements the Two-Ray Ground Reflection model. The received power P_r at horizontal distance d is:

$$P_r = P_t G_t G_r \frac{h_t^2 h_r^2}{d^4}, \tag{24}$$

where P_t is transmit power, G_t, G_r are antenna gains, and $h_t = 1.5$ m (sensor), $h_r = 100$ m (UAV). For $d < d_{\text{cross}}$ (crossover distance) the free-space Friis model is used instead. Zero-mean Gaussian shadowing $\mathcal{N}(0,\, 4\,\text{dB}^2)$ is added independently per link per timestep. The path loss exponent effectively equals 4 in the Two-Ray regime, consistent with standard outdoor LoRa deployments [28].

Reproduction

Full instructions for training the DQN agent and reproducing all evaluation results are in the repository `README.md`. All random seeds used in the evaluation ($\{42, 123, 256, 789, 1337, \ldots\}$, 20 total per condition) are hard-coded in the evaluation scripts to ensure bit-exact reproducibility.

Third-Party Code

All third-party code used in this project is clearly identified in the repository. The DQN implementation uses the Stable-Baselines3 library (MIT licence). The Gymnasium environment interface follows the standard `gymnasium.Env` API. No third-party code is incorporated into the simulation environment or reward function, which are entirely original.

G DQN Hyperparameters

Table 17. Complete DQN Hyperparameter and Training Configuration

Parameter	**Value**
Total Timesteps	2×10^6
Learning Rate (α)	3×10^{-4}
Discount Factor (γ)	0.99
Replay Buffer Size	150,000 transitions
Batch Size	128
Target Network Update (C)	every 1,000 steps
Gradient Clip (max norm)	10
Exploration Strategy	ϵ-greedy
Initial ϵ	1.0
Final ϵ	0.05
Exploration Fraction	25% of total timesteps
Policy Network Architecture	MLP [512, 512, 256]
Activation Function	ReLU
Frame Stack (k)	4
Max Sensors Limit (N_{max})	50 (zero-padded)
Number of Training Environments	4 (parallel)

Gradient clipping (max norm 10) was introduced after observing reward collapse during early curriculum training on large grids ($N = 40$) without it. The learning rate was reduced from 10^{-3} to 3×10^{-4} in the same corrective iteration, as discussed in Section 7.

Training Algorithm

Algorithm 1 Protocol-Aware DQN Training for UAV Data Collection

Initialise: UAV position p_0, Replay Buffer $\mathcal{D}$, Q-network $Q(s, a|\theta)$, target $\hat{Q}(s, a|\theta^-)$
Initialise: $\epsilon \leftarrow \epsilon_{\text{max}}$; sensor states (buffers, AoI, SF)
for episode $\in [1, M]$ **do**
    Reset environment: $s_0 \leftarrow (p_{\text{start}}, E_{\text{max}}, \ldots)$
    while $E > 0$ **do**
        $a_t \leftarrow \begin{cases} \text{random} & \text{w.p. } \epsilon \\ \arg\max_a Q(s_t, a; \theta) & \text{otherwise} \end{cases}$
        $s_{t+1}, r_t \leftarrow$ env.step(a_t); store (s_t, a_t, r_t, s_{t+1}) in $\mathcal{D}$
        if $|\mathcal{D}| \geq B$ **then**
            Sample minibatch; compute $y_j = r_j + \gamma \max_{a'} \hat{Q}(s'_j, a'|\theta^-)$
            $\theta \leftarrow \theta - \alpha \nabla_\theta \frac{1}{B} \sum_j (y_j - Q(s_j, a_j|\theta))^2$
        end if
    end while
    Decay ϵ; update $\theta^- \leftarrow \theta$ every C steps
end for

H Sim-to-Real Perturbation Results

Table 18. DQN reward and Jain's fairness index under three domain shift conditions (mean $\pm$ std, 5 seeds, $N = 20$, 500×500). Baseline is the unperturbed evaluation condition for each perturbation type.

Perturbation	Condition	Mean Reward ($\times 10^6$)	$\pm$ Std	% vs. Baseline
GPS noise (σ)	$\sigma = 0$ (baseline)	5.128	0.616	,
	$\sigma = 1$ (10 m)	5.581	0.320	$+8.8$%
	$\sigma = 5$ (50 m)	5.837	0.431	$+13.8$%
	$\sigma = 10$ (100 m)	5.826	0.434	$+13.6$%
Path-loss shift (Δn)	$\Delta n = 0$ (baseline)	5.979	,	,
	$\Delta n = +0.2$	5.331	,	-10.8%
	$\Delta n = +0.5$	5.947	,	-0.5%
	$\Delta n = +1.0$	5.939	,	-0.7%
Duty cycle (%)	10% (baseline)	5.708	,	,
	8%	5.912	,	$+3.6$%
	6%	5.211	,	-8.7%
	4%	4.897	,	-14.2%

Note on GPS results: The apparent reward increase with GPS noise reflects seed variance (std $\approx 600{,}000$, comparable to the observed differences). The increase is not statistically significant at $n = 5$ and should not be interpreted as noise improving performance. The key finding is the *absence of degradation* at realistic noise levels, confirming policy robustness.

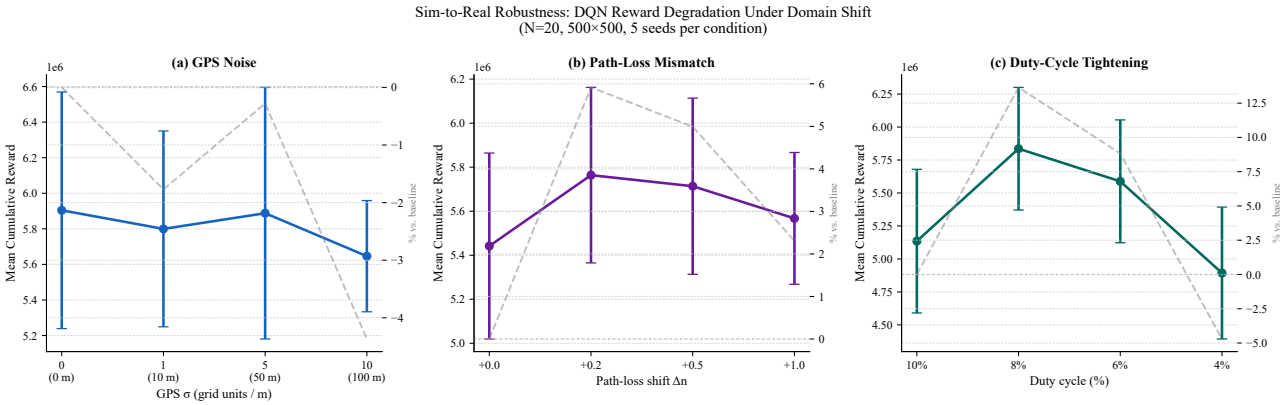


Fig. 4. DQN reward under three domain shift conditions. Left: GPS position noise. Centre: path-loss model mismatch. Right: duty-cycle tightening. Secondary y-axis shows percentage change relative to the unperturbed baseline.

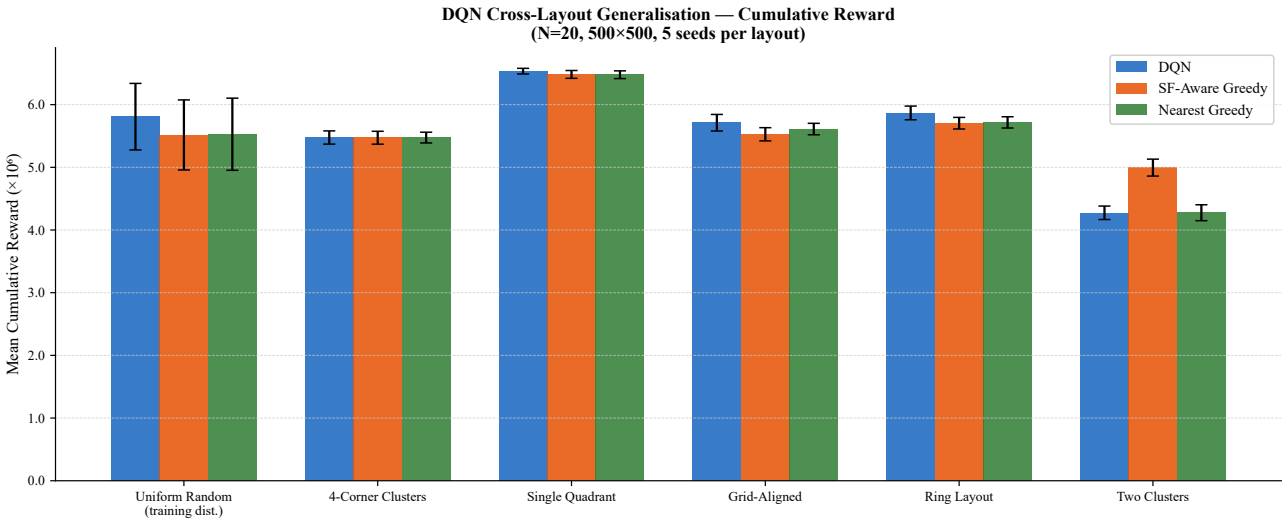


Fig. 5. Mean cumulative reward per layout type for DQN, SF-Aware Greedy, and Nearest-Sensor Greedy (5 seeds, $N = 20$, 500×500). Error bars show ± 1 standard deviation across seeds.

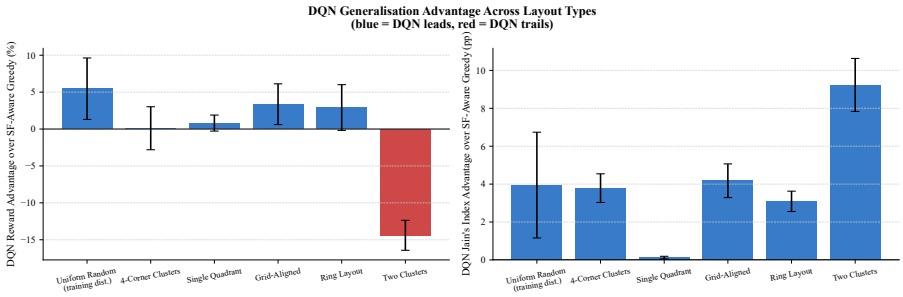


Fig. 6. DQN percentage reward advantage over SF-Aware Greedy by layout type. Significant advantages ($p < 0.05$, Welch's t) are marked with ***. The two-cluster layout is the only configuration where the DQN underperforms.

I Additional Results Figures

Training Convergence

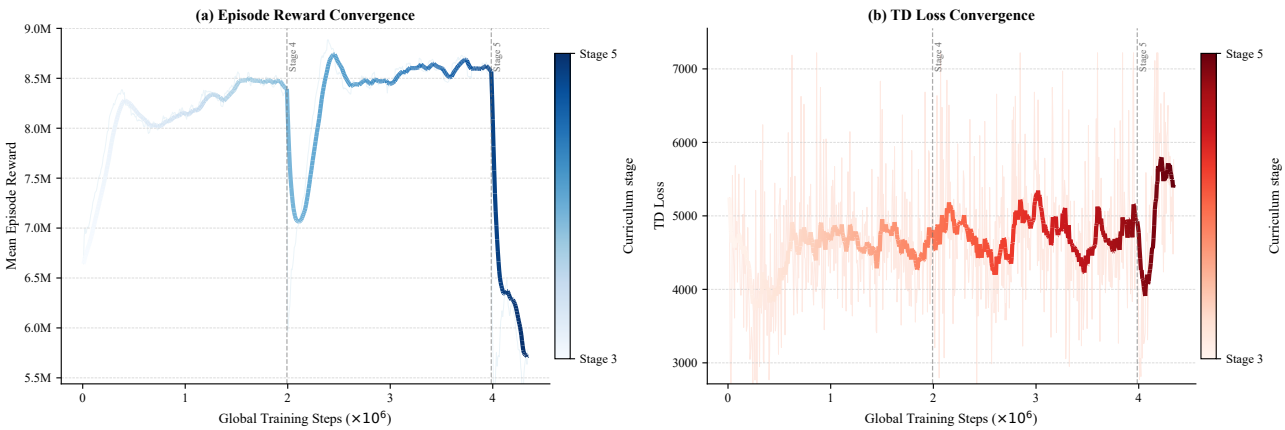


Fig. 7. Training convergence per curriculum stage (8 stages with ≥ 50 logged episodes shown; short interrupted stages omitted). Each line is one stage, coloured light-to-dark with stage index. **Left:** mean episode reward , early stages (4–6) converge at large negative values; later stages (7, 10, 13, 15, 16) converge to 3–5×10^6. **Right:** TD loss, all stages show the spike-then-converge profile. Raw values in transparent colour; EMA ($w = 0.9$) in bold.

Effect Size Summary

Table 19. Cohen's d effect sizes for DQN vs. SF-Aware Greedy (cumulative reward), 20-seed evaluation (30-seed for $N = 40$). $d < 0.2$ = negligible; 0.2–0.5 = small; 0.5–0.8 = medium; > 0.8 = large.

N	Reward d	Jain's d	Efficiency d	Interpretation
10	2.52	0.43	2.68	Large ($p < 0.001$)
20	0.66	0.36	0.67	Medium ($p = 0.049^*$)
30	0.78	0.61	0.77	Medium ($p = 0.021^*$)
40	0.54	0.40	0.55	Medium ($p = 0.043^*$)

Single-Episode Reward Curve

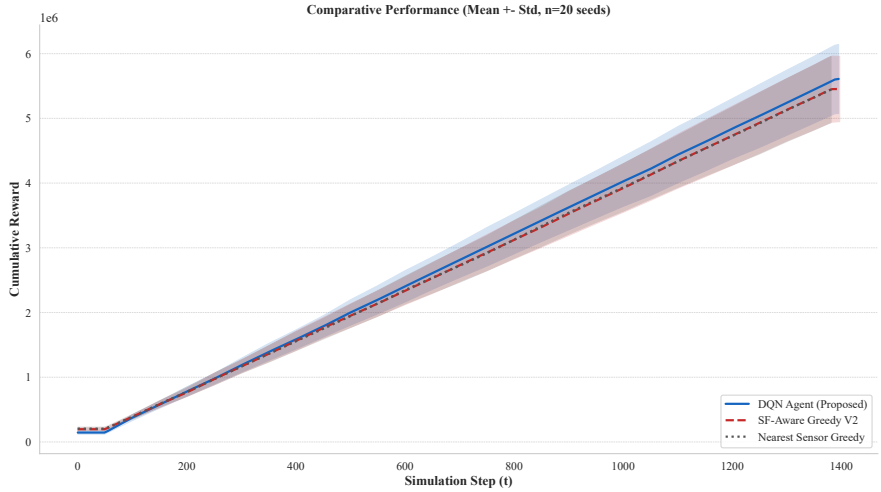
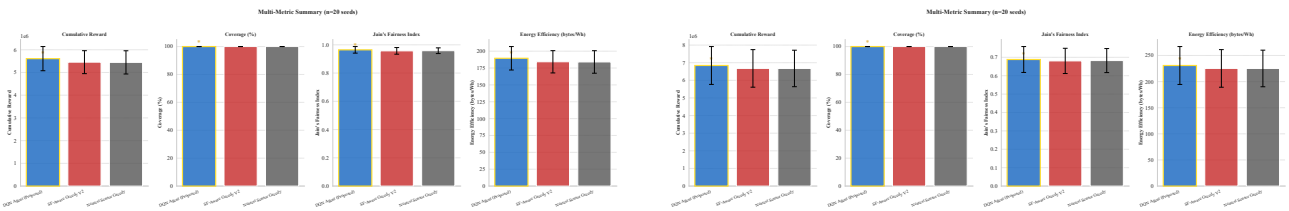


Fig. 8. Mean cumulative reward (± 1 std, $n = 20$ seeds) for the trained DQN agent (20 sensors, 500$\times$500 grid). The shaded band shows inter-seed variance; battery depletion occurs at approximately step 1,400.

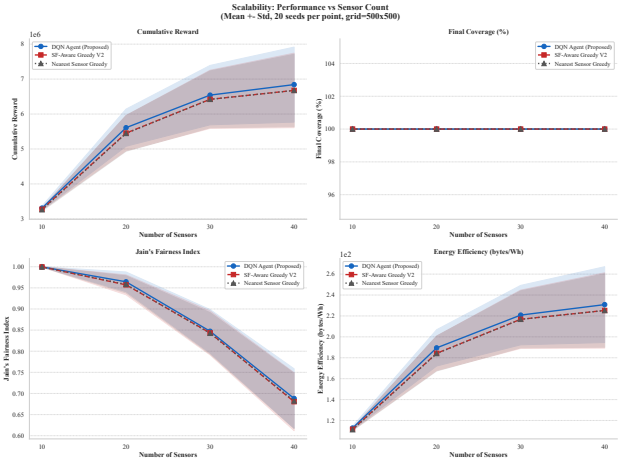
Multi-Seed Comparative Figures



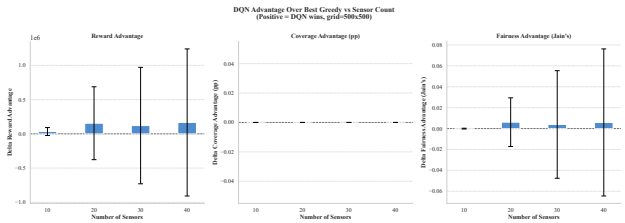
(a) $N = 20$ sensors. **(b)** $N = 40$ sensors.

Fig. 9. Per-metric bar charts (mean $\pm$ std across twenty seeds) comparing DQN and greedy baselines at two representative network densities.

Scalability Figures

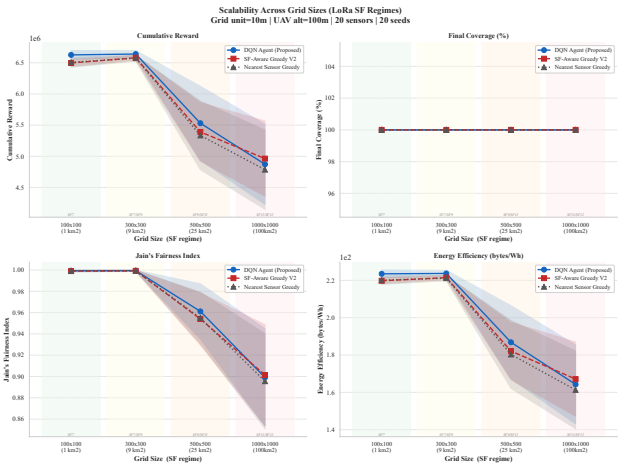


(a) Absolute metric values vs. sensor count.

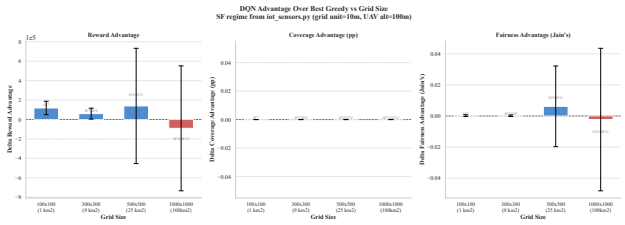


(b) DQN advantage (%) over SF-Aware Greedy.

Fig. 10. Scalability of the three agents as a function of network density ($N \in \{10, 20, 30, 40\}$, 500×500 grid, twenty seeds each).



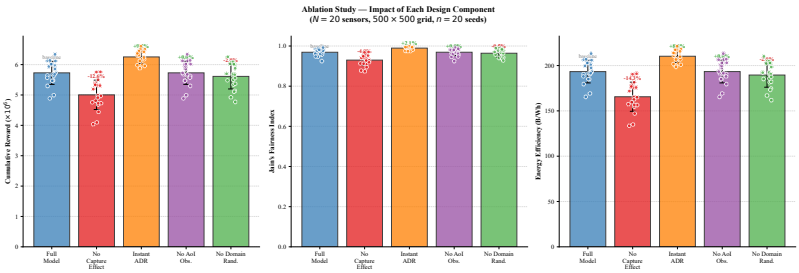
(a) Absolute metric values vs. grid size.



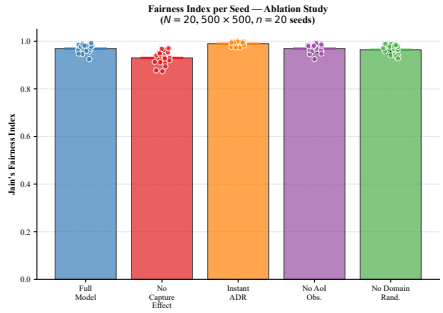
(b) DQN advantage (%) over SF-Aware Greedy.

Fig. 11. Scalability of the three agents as a function of grid size ($N = 20$ sensors, twenty seeds each).

Ablation Study Figures



(a) Mean $\pm$ std for cumulative reward, Jain's index, and energy efficiency across all ablation conditions.



(b) Per-seed Jain's index scatter. Horizontal bars show the mean.

Fig. 12. Ablation study results ($N = 20$, 500$\times$500 grid, $n = 20$ seeds). Blue = Full Model.

Spreading Factor Dynamics

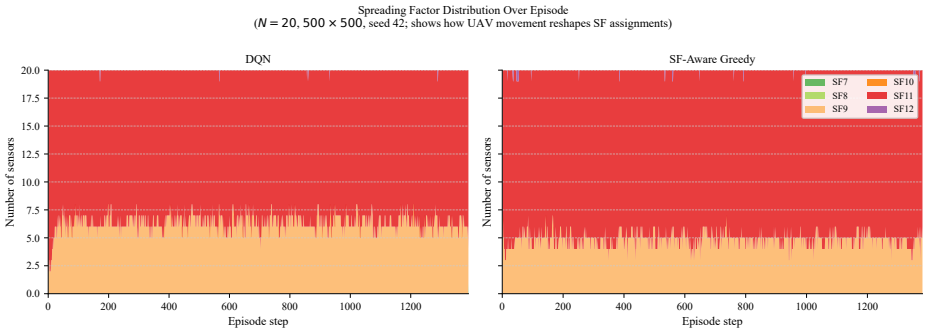


Fig. 13. Spreading factor distribution across all 20 sensors over the episode (seed 42). The DQN (left) maintains greater SF diversity and achieves faster SF downgrades than the SF-Aware Greedy baseline (right), confirming that deliberate repositioning actively reshapes the LoRaWAN spectral environment.

HIL Preparation Analysis

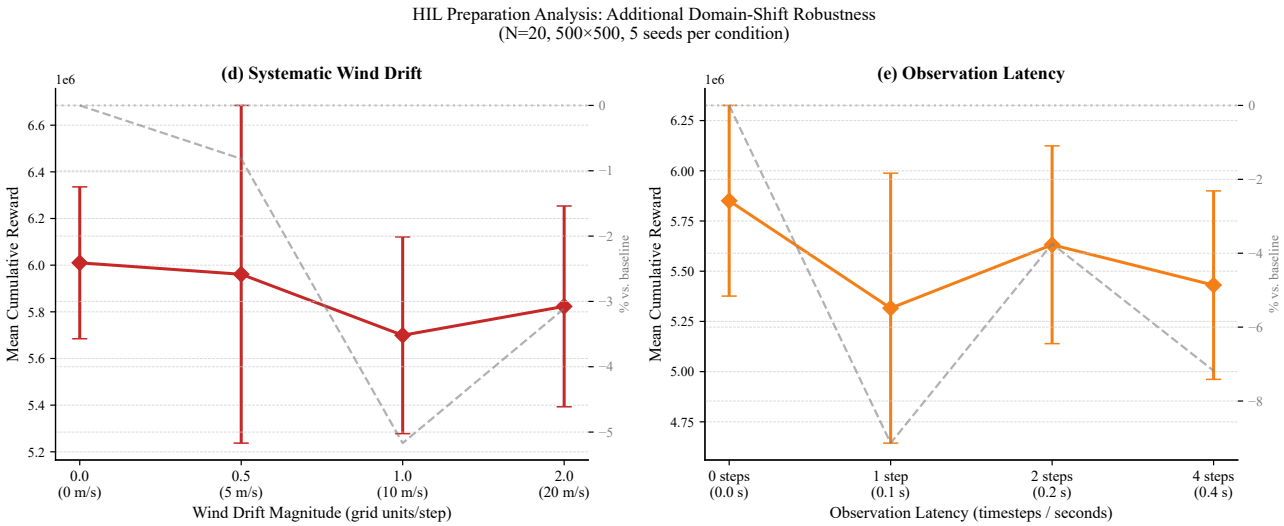


Fig. 14. HIL preparation analysis: DQN reward under systematic wind drift (left) and observation latency (right). Error bars show ± 1 std across 5 seeds. Secondary y-axis shows percentage change relative to the unperturbed baseline.